

Création d'une API avec le langage de requête GraphQL

Bénédicta MANGBA & François TOYI

30 avril 2025

Table des matières

1	Introduction à GraphQL	3
1.1	Qu'est-ce que GraphQL ?	3
1.2	Concepts de base de GraphQL	3
1.3	Les types en GraphQL	3
1.4	Avantages de GraphQL	6
1.5	Limites de GraphQL	6
2	Prérequis pour utiliser GraphQL sur Ubuntu	6
2.1	Installation des outils nécessaires	6
2.2	Passons à la pratique! 🐦	6
3	Configuration de l'environnement de travail	6
4	Initialisation du projet	7
4.1	Commandes de base et explication	7
5	Modification du fichier package.json	7
6	Installation des dépendances	8
6.1	Commandes de base et explication	8
7	Démarrage du serveur dans le fichier index.js	9
8	Exécution du serveur	10
9	Test pratique	11
9.1	Ajout de nouveaux types et requêtes 🛠️📊	11
10	Fin de la section de test	22

11	Projet réaliste : Gestion des contacts avec Prisma et Flutter	23
11.1	Concept	23
11.2	Restructuration du projet ■	23
12	Configuration de Prisma en ligne de commande	24
13	Nous allons aborder le concept de mutation (create, update, delete)	37
14	Implémentation de l'authentification avec JWT et BCRYPTJS	44
14.1	Définition du Schéma GraphQL	44
14.2	Modèle User	45
14.3	Processus de la Mutation <code>register</code>	45
14.4	Processus de la Mutation <code>login</code>	45
14.5	Gestion du Token JWT	45
14.6	Sécurisation des Résolveurs	45
14.7	Résumé du Processus	46
15	Consommation de l'API GraphQL créée	61
16	Consommation de l'API GraphQL créée	61
17	Exemple d'utilisation de la requête de login dans notre projet React Native	67

1 Introduction à GraphQL

1.1 Qu'est-ce que GraphQL ?

GraphQL est un langage de requêtes pour les APIs développé par Facebook en 2015. Contrairement aux APIs REST traditionnelles, qui fonctionnent par plusieurs points de terminaison (endpoints) pour des ressources spécifiques, GraphQL utilise un seul endpoint et permet aux clients de définir précisément les données qu'ils souhaitent recevoir. Cela rend GraphQL particulièrement flexible et optimisé pour réduire le sur- ou sous-chargement de données, ce qui est très utile dans les applications modernes où le front-end peut nécessiter un contrôle plus fin sur les données échangées.

1.2 Concepts de base de GraphQL

- **Schéma et types** : Le cœur de GraphQL repose sur un schéma fort typé. Un schéma définit les types d'objets que l'API expose, leurs champs et leurs relations.
- **Requêtes (queries)** : Les requêtes permettent aux clients de demander des données spécifiques en fonction de leurs besoins.
- **Mutations** : Les mutations permettent d'envoyer des modifications au serveur, comme la création, la mise à jour, ou la suppression d'enregistrements.
- **Résolveurs** : Un résolveur est une fonction qui gère la logique derrière chaque champ de la requête.
- **Souscriptions** : Les souscriptions permettent de recevoir des mises à jour en temps réel, utiles dans les applications en temps réel.

1.3 Les types en GraphQL

- **Types scalaires** : Ce sont les types de base utilisés pour représenter des valeurs simples. Les principaux types scalaires sont :
 - **Int** : Nombre entier.
 - **Float** : Nombre décimal.
 - **String** : Chaîne de texte.
 - **Boolean** : Valeur booléenne (**true** ou **false**).
 - **ID** : Identifiant unique.
- **Types objets** : Ils définissent des structures de données regroupant plusieurs champs.
- **Types racines** : GraphQL repose sur trois types principaux permettant l'interaction avec l'API :
 - **Query** : Permet de récupérer des données.
 - **Mutation** : Permet de modifier des données (ajout, suppression, mise à jour).
 - **Subscription** : Permet de recevoir des mises à jour en temps réel.
- **Types avancés** :

- `[Type]` : Représente une liste d'éléments d'un type donné.
- `Type!` : Indique qu'une valeur est obligatoire (non nulle).
- `enum` : Définit un ensemble de valeurs prédéfinies.
- `interface` : Permet de définir des structures communes à plusieurs types.



Les types GraphQL avec des simpleexemples

1. Types scalaires *

- Int : Nombre entier Ex : 42
- Float : Nombre décimal Ex : 3.14
- String : Texte Ex : "Bonjour ! ☺"
- Boolean : Vrai/Faux Ex : true
- ID : Identifiant unique Ex : "abc123"

2. Types objets 📦 (Groupes de données)

```

1
2 type Livre {
3   titre: String!
4   pages: Int!
5   disponible: Boolean!
6 }

```

3. Types racines >_ (Points d'entrée)

```

1 # Lire des donnees
2 type Query {
3   livres: [Livre!]!
4 }
5
6 # Modifier des donnees
7 type Mutation {
8   ajouterLivre(titre: String!): Livre!
9 }
10
11 # Recevoir en temps reel
12 type Subscription {
13   nouveauLivre: Livre!
14 }

```

4. Types avancés 🧩

- [Type] : Liste Ex : [String]
 - Type! : Obligatoire Ex : Int!
 - enum : Choix fixés Ex : enum Couleur ROUGE VERT BLEU
 - interface : Structure commune Ex : interface Produit
- prix: Float!

Exemple complet </>

```

1 type Query {
2   produits: [Produit!]!
3 }
4
5 type Produit {
6   id: ID!
7   nom: String!
8   prix: Float!
9   enStock: Boolean!
10 }

```

1.4 Avantages de GraphQL

- [Précision des requêtes](#) : Les clients peuvent demander uniquement les champs dont ils ont besoin.
- [Un endpoint unique](#) : Simplifie la gestion des APIs, puisque toutes les opérations passent par un seul point de terminaison.
- [Performance optimisée](#) : Moins de surcharges de données et plus de flexibilité dans le traitement des informations.
- [Documentation intégrée](#) : GraphQL est auto-documenté, ce qui facilite la découverte des possibilités de l'API.

1.5 Limites de GraphQL

- [Complexité](#) : Il peut être complexe à configurer et à apprendre, notamment pour les grands schémas.

2 Prérequis pour utiliser GraphQL sur Ubuntu

2.1 Installation des outils nécessaires

Avant de commencer à utiliser GraphQL, vous devez installer certains outils sur votre machine Ubuntu. Voici les étapes à suivre :

1. [Node.js et npm](#) : GraphQL est souvent utilisé avec Node.js. Pour installer Node.js et npm (Node Package Manager), exécutez les commandes suivantes :

```
1 sudo apt update
2 sudo apt install nodejs npm
3
```

2. [Éditeur de texte](#) : Vous aurez besoin d'un éditeur de texte pour écrire votre code. Vous pouvez utiliser [Visual Studio Code](#) (VS Code) qui est très populaire parmi les développeurs. Pour installer VS Code :

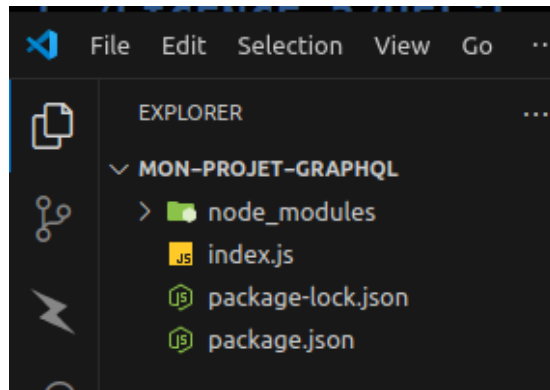
```
1 sudo snap install --classic code
2
```

2.2 Passons à la pratique! 🐦

Maintenant que tout est installé, nous allons créer un projet GraphQL simple. Suivez les étapes ci-dessous pour afficher "Hello World" avec GraphQL. Création d'un projet GraphQL

3 Configuration de l'environnement de travail

Maintenant que tout est installé, nous allons créer un projet GraphQL simple. Suivez les étapes ci-dessous pour afficher "Hello World" avec GraphQL. Voici à quoi ressemblerait notre structure :

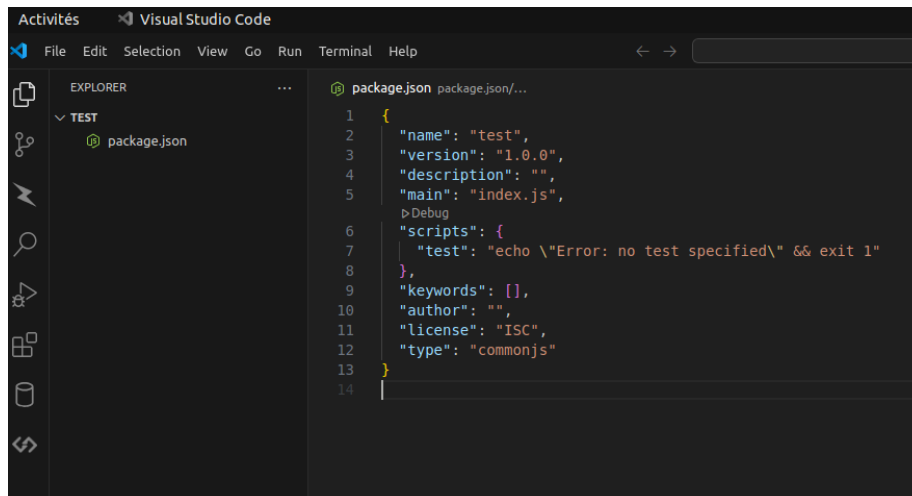


4 Initialisation du projet

4.1 Commandes de base et explication

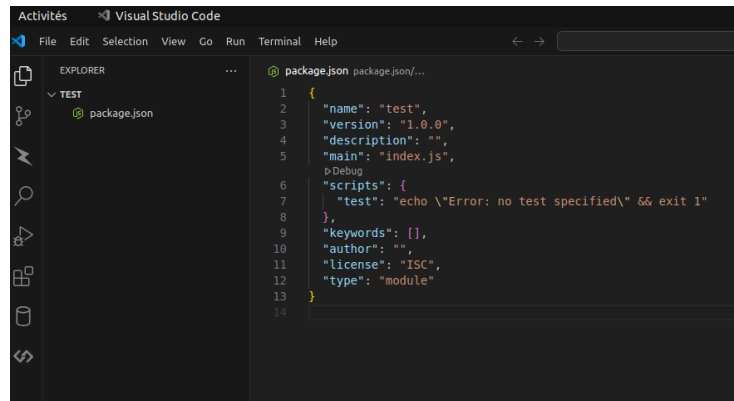
```
1 mkdir test // Creation du dossier du projet
2 cd test // Acces au dossier du projet
3 npm init -y // Initialisation dun projet Node.js
```

Après cette commande, vous aurez un fichier `package.json` qui ressemblera à ceci :



5 Modification du fichier package.json

Maintenant, vous devez changer le type de module de `commonjs` en `module`, car c'est ce que nous allons utiliser dans notre projet. Modifiez la valeur de `type` de `commonjs` en `module`.



6 Installation des dépendances

6.1 Commandes de base et explication

Accéder à votre terminal et exécutez les commandes suivantes

```

1 npm install @apollo/server graphql // Installation des
  dépendances
2 npm install graphql-tag // Installation de la bibliothèque
  graphql-tag
3 npm install nodemon // pour permettre le redémarrage à chaud
  du serveur

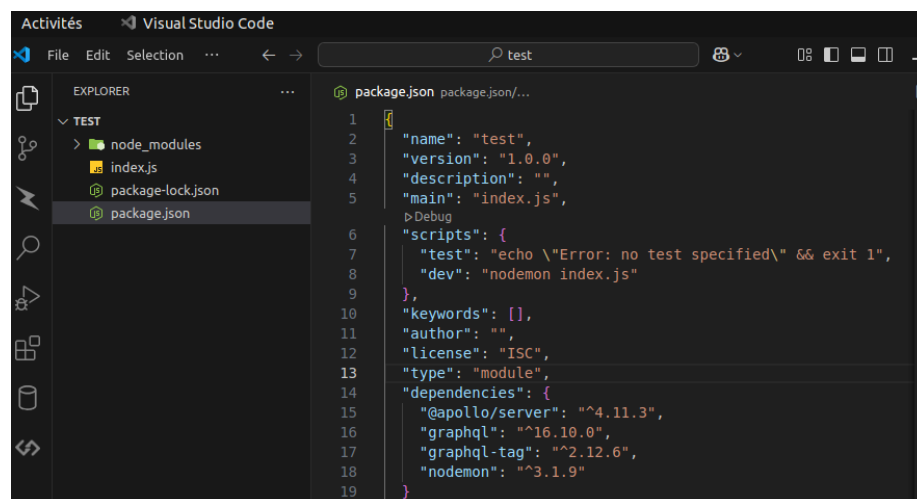
```

Dans le fichier `package.json`, au niveau de la section `script`, mettez virgule(,) et ajoutez la ligne suivante :

```

1 "dev": "nodemon index.js"

```



7 Démarrage du serveur dans le fichier index.js

```
1 import { ApolloServer } from '@apollo/server';
2 import { gql } from 'graphql-tag';
3 import { startStandaloneServer } from "@apollo/server/standalone";
4
5 // Definition du schema GraphQL
6 const typeDefs = gql`
7   type Query {
8     hello: String
9   }
10 `;
11
12 // Resolveur pour le schema
13 const resolvers = {
14   Query: {
15     hello: () => 'Hello, world!',
16   },
17 };
18
19 // Creer une instance d'ApolloServer
20 const server = new ApolloServer({
21   typeDefs,
22   resolvers,
23 });
24
25 // Lancer le serveur en utilisant startStandaloneServer
26 const { url } = await startStandaloneServer(server, {
27   listen: { port: 4000 },
28 });
29
30 console.log(`Server started at ${url}graphql`);
```

Voici une explication détaillée du code index.js :

Explication du code Apollo Server

Structure du code en 5 parties :

1. **Importations des dépendances :**
 - ApolloServer : Framework GraphQL
 - gql : Parseur de schéma GraphQL
 - startStandaloneServer : Serveur HTTP intégré

2. **Définition du schéma GraphQL (typeDefs) :**

```
1   type Query {  
2     hello: String  #Expose une requete 'hello'  
3   }  
4
```

3. **Résolveurs (resolvers) :**
 - Implémente la logique de la requête
 - Renvoie toujours "Hello, world!"
4. **Création du serveur :**
 - Combine schéma et résolveurs
 - Configuration minimale
5. **Lancement du serveur :**
 - Port 4000 par défaut
 - Message de confirmation au démarrage

Fonctionnement global :

Le serveur répondra à une requête GraphQL { hello } avec la chaîne "Hello, world!".

8 Exécution du serveur

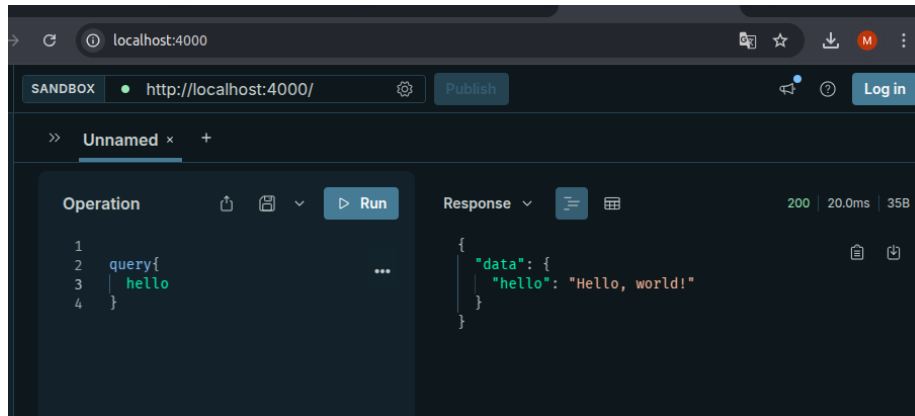
Dans votre terminal, exécutez la commande suivante pour démarrer le serveur :

```
1 npm run dev
```

Vous devriez voir le message suivant dans votre terminal :

```
yetnam@yetnam:~/Documents/LICENCE_3/UELibre/finalisation_cours_a_ramasser/test$ npm run dev  
 > test@1.0.0 dev  
 > nodemon index.js  
  
[nodemon] 3.1.9  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node index.js`  
Server ready at http://localhost:4000/  
█
```

Si vous obtenez ce résultat, cela signifie que vous avez bien effectué les tests. Ensuite, appuyez sur **Ctrl** puis cliquez sur le endpoint pour afficher votre requête. Vous n'avez qu'à taper dans la partie opération exactement ce qui est affiché sur l'image suivante pour obtenir le résultat :



9 Test pratique

9.1 Ajout de nouveaux types et requêtes ✎ 📊

Défi mathématique !

*"Et si on équipait notre API de super circuits intégrés ?
Transformons ce serveur en calculatrice GraphQL ! 🦾"*

💡 Mission :

- `</>` Créer une requête calcul pour additionner deux nombres
- 🏗️ Déclarer les types GraphQL appropriés
- ⚙️ Résolveur effectuant $a + b$

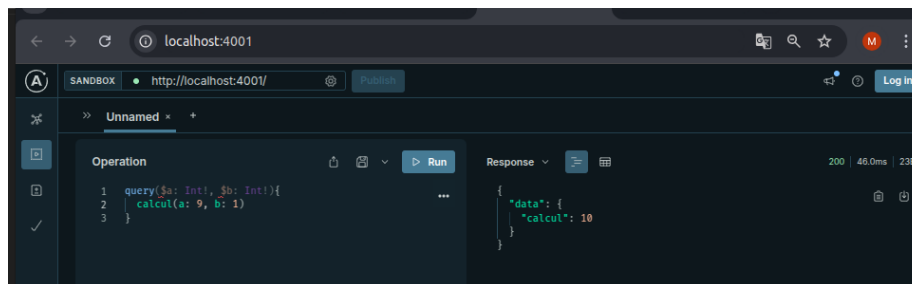
```
1
2 import { ApolloServer } from '@apollo/server';
3 import { gql } from 'graphql-tag';
4 import { startStandaloneServer } from "@apollo/server/standalone";
5
6 // Definition du schema GraphQL
7 const typeDefs = gql`
8   type Query {
9     hello: String
10    calcul(a: Int!, b: Int!): Int
11  }
12 `;
13
14 // Resolveur pour le schema
15 const resolvers = {
16   Query: {
17     hello: () => 'Hello, !',
```

```

18     calcul: (parent, args, context, info) => {
19         return args.a + args.b;
20     },
21 },
22 };
23
24 // Créer une instance d'ApolloServer
25 const server = new ApolloServer({
26     typeDefs,
27     resolvers,
28 });
29
30 // Lancer le serveur en utilisant startStandaloneServer
31 const { url } = await startStandaloneServer(server, {
32     listen: { port: 4000 },
33 });
34
35 console.log('Server started at ${url}graphql');
36

```

- ✓ Type Query avec calcul(a: Int!, b: Int!): Int!
- ✓ Résolveur qui manipule a et b
- ✓ Test avec { calcul(a: 9, b: 1) }



👍 Résultat attendu : 10(pas 57!) 😊

🗨 Astuce :

"Si votre code retourne 57..."

C'est un concaténateur masqué! 🔍

Reprenons notre sérieux et comprenons le code suivant, cela nous est très important :

Structure du code

1. Schéma GraphQL :

```
1 type Query {  
2   "Ajoute deux entiers (a + b)"  
3   calcul(a: Int!, b: Int): Int  
4 }
```

- `Int!` : Paramètre obligatoire
- `Int` : Peut retourner null

2. Résolveur JavaScript :

```
1 const resolvers = {  
2   // 1. Requête  
3   Query: {  
4     calcul: (_, args) => {  
5  
6       // 2. Calcul  
7       const result = args.a + args.b;  
8  
9       // 3. Retour  
10      return result;  
11    }  
12  }  
13 };
```

💡 Explications clés

- ➡ `_` Placeholder pour l'objet parent
- ➡ `args` { a: 9, b: 1 } (arguments)
- ✂ **Retour** Doit matcher le type GraphQL

🎬 Flux d'exécution

1. Requête client : `query { calcul(a:9, b:1) }`
2. Vérification des types par GraphQL
3. Exécution du résolveur si valide
4. Retour : `{ "data": { "calcul": 10 } }`

Alors maintenant, nous allons tester les types objets.

Pour commencer, les types objets sont des types personnalisés.
Voici ce que nous allons ajouter dans le fichier `index` :
Nous allons définir une tâche sur laquelle nous allons effectuer un petit CRUD. Voici les étapes à suivre :

1. Définition du type Tache

Pour définir le type, voici ce que nous allons ajouter dans le `typedef` :

```
1 type Tache {
2   id: ID!
3   titre: String!
4   terminer: Boolean!
5 }
```

2. Définition de la requête

Ensuite, nous allons définir la requête pour obtenir la liste des tâches :

```
1 taches: [Tache]
```

3. Définition du resolveur

Enfin, nous allons définir le resolveur qui va gérer la requête des tâches :

```
1 taches: () => tasks
```

4. Exemple de données

Voici un exemple de données dans une liste pour effectuer les tests :

```
1 let tasks = [
2   { id: 1, titre: "Apprendre GraphQL", terminer: false },
3   { id: 2, titre: "Faire les courses", terminer: true }
4 ];
```

Ces étapes vous permettront de définir et d'utiliser des types objets dans GraphQL pour effectuer un CRUD basique.

Votre fichier `index.js` devrait ressembler à ceci

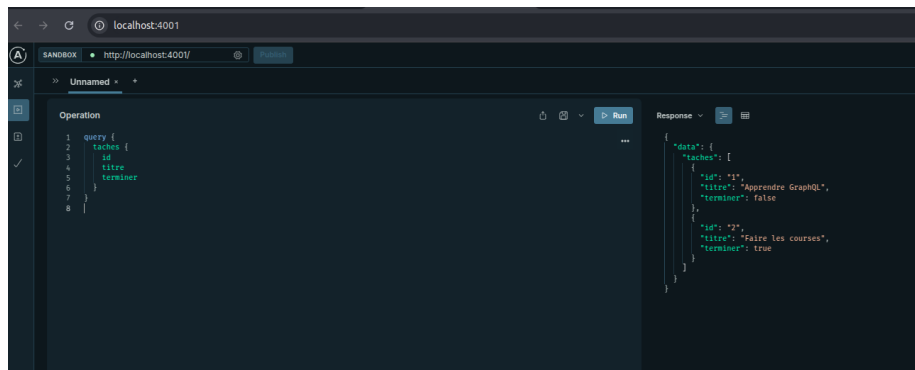
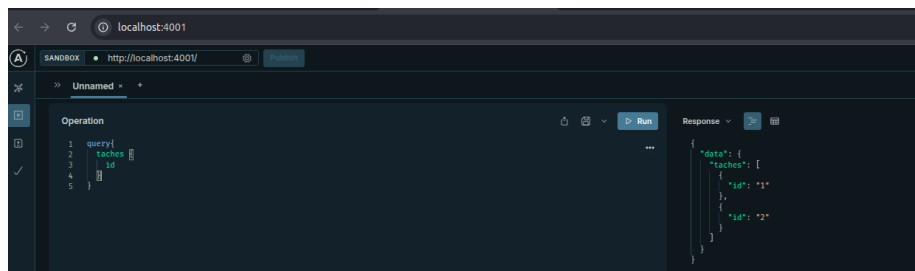
```
1 import { ApolloServer } from '@apollo/server';
2 import { gql } from 'graphql-tag';
3 import { startStandaloneServer } from "@apollo/server/standalone";
4
5 // Définition du schéma GraphQL
6 const typeDefs = gql`
7   type Tache {
8     id: ID!
9     titre: String!
10    terminer: Boolean!
11  }
12
13  type Query {
14    hello: String
15    calcul(a: Int!, b: Int!): Int
16    taches: [Tache]
```

```

17   }
18   ';
19
20   let tasks = [
21     { id: 1, titre: "Apprendre GraphQL", terminer: false },
22     { id: 2, titre: "Faire les courses", terminer: true }
23   ];
24
25   // Resolvants pour le schéma
26   const resolvers = {
27     Query: {
28       hello: () => 'Hello, world!',
29       calcul: (_, args) => args.a + args.b,
30       taches: () => tasks
31     }
32   };
33
34   // Créer une instance d'ApolloServer
35   const server = new ApolloServer({
36     typeDefs,
37     resolvers,
38   });
39
40   // Lancer le serveur en utilisant startStandaloneServer
41   const { url } = await startStandaloneServer(server, {
42     listen: { port: 4000 },
43   });
44
45   console.log(`Server started at ${url}graphql`);

```

Maintenant pour tester retourner sur le playground et faite :



Ce qui rend GraphQL si puissant ?

(On vous l'avait déjà expliqué, mais un rappel est toujours utile)

Avec GraphQL, **personnalisez à la volée** les données affichées !

Plus de surcharge, place à la précision

Vous décidez des champs, rien que ceux dont vous avez besoin

Exactement ce que vous voulez, rien de plus

(Une flexibilité que REST ne peut égaler!)

Maintenant , faisons la mise à jour , la creation et la supression, nous pourrons maintenant manipuler notre type **Mutation**

Pour se faire, nous allons utiliser le type **Tache** déjà défini , puis d'abord déclarer la mutation apres cela nou allons de definir le corps de la mutation dans le resolvers . Voici ce qu'il faut faire :

```
1
2  type Tache {
3      id: ID!
4      titre: String!
5      terminer: Boolean!
6  }
```

Declarer la mutation

```
1
2  type Mutation{
3      ajouterTache(titre: String!, terminer: Boolean!): Tache
4
5  }
```

Apres definir le corps de la mutation dans le resolveur


```

1
2 Mutation:{
3   ajouterTache: (_, {titre, terminer}) => {
4     const nouvelleTache = {
5       id: tasks.length + 1,
6       titre,
7       terminer
8     };
9     tache.push(nouvelleTache);
10    return nouvelleTache;
11  }
12 }

```

Si vous faite bien ce qui a été dit; Voici la manière dont votre fichier index devrait etre

```

1 import { ApolloServer } from '@apollo/server';
2 import { gql } from 'graphql-tag';
3 import { startStandaloneServer } from "@apollo/server/standalone";
4
5 // Definition du schema GraphQL
6 const typeDefs = gql`
7   type Tache {
8     id: ID!
9     titre: String!
10    terminer: Boolean!
11  }
12  type Query {
13    hello: String
14    calcul(a: Int!, b: Int!): Int
15    taches: [Tache]
16  }
17  type Mutation{
18    ajouterTache(titre: String!, terminer: Boolean!): Tache
19  }
20 `;
21 let tasks = [
22   { id: 1, titre: "Apprendre GraphQL", terminer: false },
23   { id: 2, titre: "Faire les courses", terminer: true }
24 ];
25 // Resolvants pour le schema
26 const resolvers = {
27   Query: {
28     hello: () => 'Hello, world!',
29     calcul: (_, args) => args.a + args.b,
30     taches: () => tasks,
31   },
32   Mutation:{
33     ajouterTache: (_, { titre, terminer }) => {
34       const nouvelleTache = {
35         id: taches.length + 1,
36         titre,
37         terminer
38       };
39       taches.push(nouvelleTache);
40       return nouvelleTache;

```

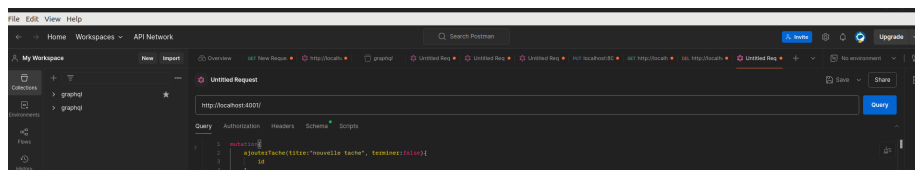
```

41   }
42 }
43 };
44 // Créer une instance de ApolloServer
45 const server = new ApolloServer({
46   typeDefs,
47   resolvers,
48 });
49
50 // Lancer le serveur en utilisant startStandaloneServer
51 const { url } = await startStandaloneServer(server, {
52   listen: { port: 4000 },
53 });
54
55 console.log(`Server started at ${url}graphql`);

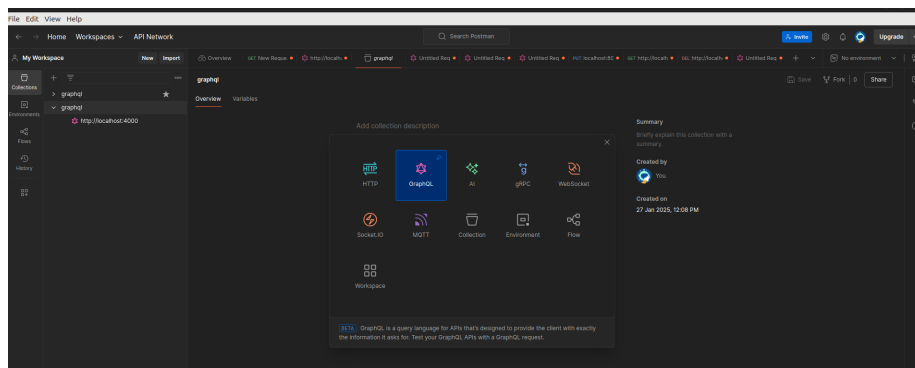
```

Maintenant que nous avons terminé la définition de notre mutation, pourquoi ne pas tester tout ça ? Il est important de noter que GraphQL ne s'exécute pas uniquement dans le Playground, mais aussi dans des clients HTTP comme Postman. Voici comment vous pouvez tester la création d'une tâche dans Postman.

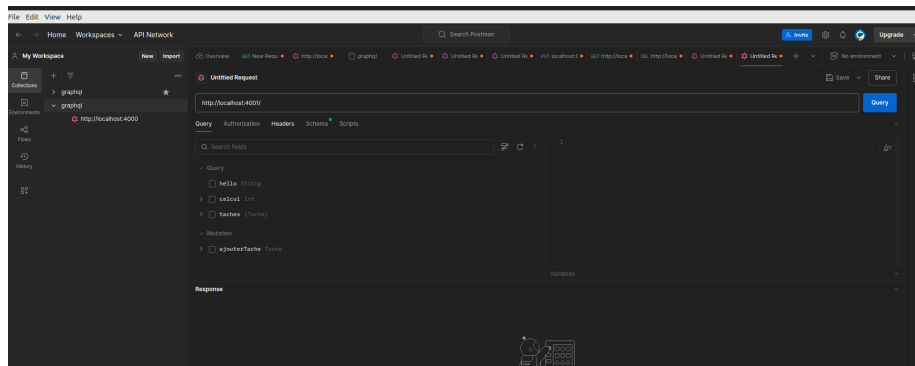
D'abord, téléchargez Postman en suivant ce lien : [Télécharger Postman](#). Une fois l'installation terminée, si vous ne l'avez pas encore configuré, voici comment procéder :



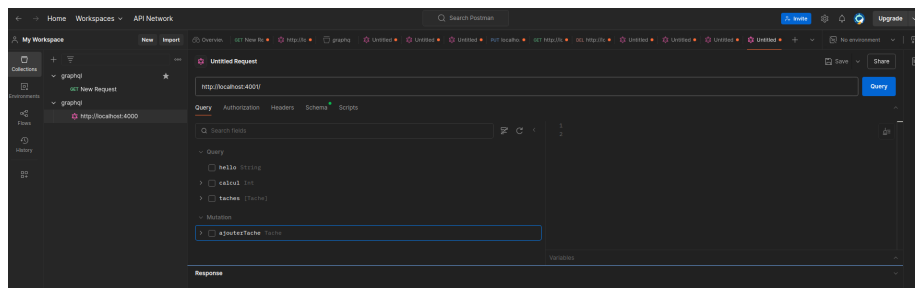
Ensuite, en haut à gauche, là où il est écrit New et Import, cliquez sur New pour ouvrir un nouvel onglet. Sélectionnez ensuite GraphQL. Voici l'illustration :



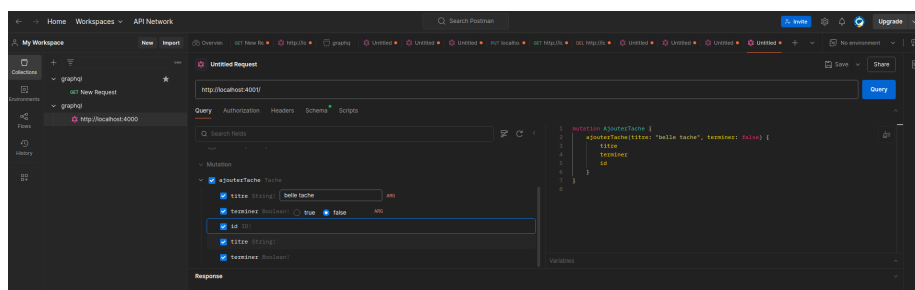
Une fois que vous avez cliqué dessus, entrez l'URL de votre endpoint unique en haut, comme montré ci-dessous. N'oubliez pas de remplacer l'exemple par le vôtre.



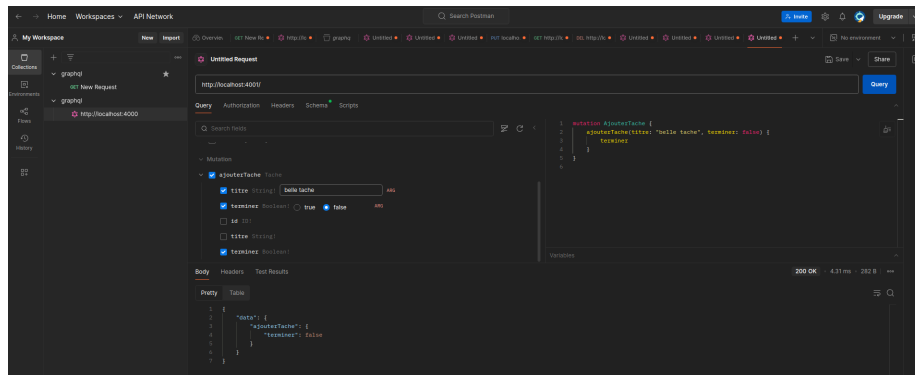
Maintenant que tout est prêt, voici comment tester la création d'une tâche dans Postman. Une fois que vous accédez à Postman, vous verrez la liste de vos requêtes et mutations, comme montré ci-dessous :



Cliquez sur le nom de votre mutation. Vous aurez alors des champs pour saisir les données nécessaires. Voici l'avantage d'utiliser Postman par rapport au Playground.



Par défaut, tous les champs sont sélectionnés, ce qui signifie que tous seront retournés après l'envoi des données. Ne vous inquiétez pas, lors de la définition de la mutation en GraphQL, nous avons précisé le type de retour. Vous pouvez donc choisir les champs que vous souhaitez recevoir. En cliquant sur Query, voici le résultat que vous obtiendrez :



Faisons également la suppression :

Même scénario : nous allons définir le schéma :

```
1 supprimerTache(id: ID!): Tache
```

Et pour finir définir le résolveur :

```
2 supprimerTache: (_, {id}) => {
3   const supTask = tasks.find((task) => task.id == id)
4   tasks = tasks.filter((task) => task.id != id);
5   return supTask;
6 }
```

Voici à quoi devrait ressembler votre index. Vous remarquerez que la méthode de lancement du serveur a changé. Sachez que ces deux codes accomplissent la même tâche. Nous avons décidé d'utiliser cette nouvelle méthode, car elle est plus recommandée et plus compréhensible.

Si vous le souhaitez, vous pouvez conserver l'ancienne méthode ou essayer cette nouvelle approche.

```
7
8
9 import { ApolloServer } from '@apollo/server';
10 import { gql } from 'graphql-tag';
11 import { startStandaloneServer } from '@apollo/server/
   standalone';
12
13 // Definition du schema GraphQL
14 const typeDefs = gql`
15   type Tache {
16     id: ID!
17     titre: String!
18     terminer: Boolean!
19   }
20
21   type Query {
22     hello: String
```

```

23     calcul(a: Int, b: Int): Int
24     taches: [Tache]
25   }
26
27   type Mutation {
28     ajouterTache(titre: String!, terminer: Boolean!): Tache,
29     supprimerTache(id: ID!): Tache
30   }
31   ‘;
32
33   let tasks = [
34     { id: 1, titre: 'Apprendre GraphQL', terminer: false },
35     { id: 2, titre: 'Faire les courses', terminer: true }
36   ];
37
38   // Resolvants pour le schema
39   const resolvers = {
40     Query: {
41       hello: () => 'Hello, world!',
42       calcul: (_, args) => args.a + args.b,
43       taches: () => tasks,
44     },
45
46     Mutation: {
47       ajouterTache: (_, { titre, terminer }) => {
48         const nouvelleTache = {
49           id: taches.length + 1,
50           titre,
51           terminer
52         };
53         taches.push(nouvelleTache);
54         return nouvelleTache;
55       },
56
57       supprimerTache: (_, {id}) => {
58         const suppTask = tasks.find((task) => task.id == id)
59         tasks = tasks.filter((task) => task.id != id);
60         return suppTask;
61       }
62     }
63   };
64
65   // Creer une instance d'ApolloServer
66   const server = new ApolloServer({
67     typeDefs,
68     resolvers,
69   });
70
71   // Lancer le serveur en utilisant startStandaloneServer
72   const { url } = await startStandaloneServer(server, {

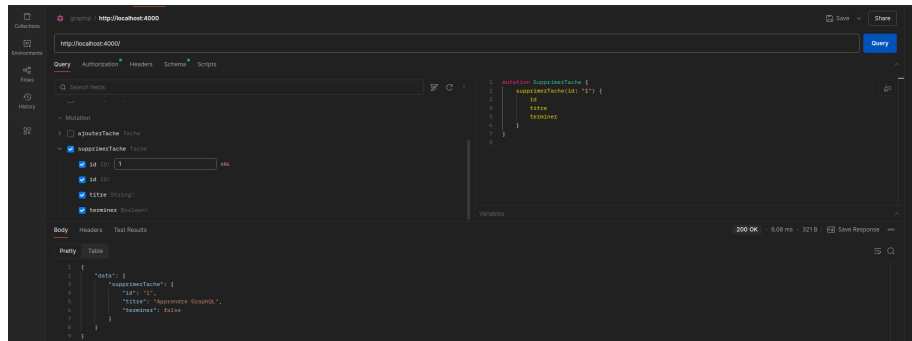
```

```

73     ecoute: { port: 4000 },
74   });
75   console.log('Server start at ${url}graphql ');

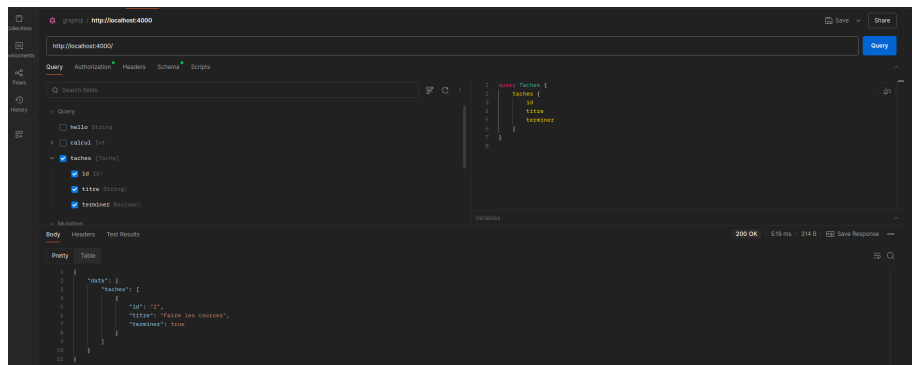
```

Voici le resultat que vous devez avoir :



Vous remarquerez que le champ de saisie de l'ID s'affiche correctement. Lorsque l'ID est saisi puis supprimé, le système renvoie bien la tâche supprimée, car le retour attendu est de type tâche.

De plus, en réaffichant la liste des tâches, on constate que la tâche ayant l'ID 1 a bien été supprimée, car elle ne s'affiche plus.



À vous de manipulez pour plus de compréhension !.

10 Fin de la section de test

Félicitations! 🎉 Vous avez réussi à créer votre première API GraphQL et à effectuer des tests pratiques. Dans la prochaine section, nous explorerons des concepts plus avancés, tels que les souscriptions, l'intégration avec une base de données, et bien d'autres sujets passionnants 😊.

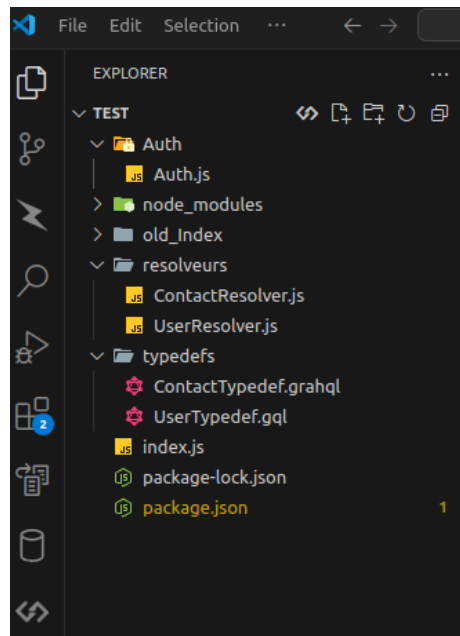
11 Projet réaliste : Gestion des contacts avec Prisma et Flutter

11.1 Concept

Maintenant que vous avez une compréhension de base de GraphQL, nous allons passer à un projet plus réaliste. Nous allons créer une API GraphQL pour gérer une liste de contacts, en utilisant Prisma comme ORM pour interagir avec une base de données, JWT pour la gestion de l'authentification. Plus tard, nous consommerons cette API.

11.2 Restructuration du projet 📁

Voici à quoi doit ressembler la structure du projet :
Reproduisez minutieusement l'architecture suivante pour structurer proprement notre projet.



📁Dossiers

Les dossiers créés seront :

- **resolvers**
- **typedefs**
- **auth**
- **old_index**

Créer les fichiers se trouvant sur la capture respectivement dans chaque dossier.

■ Sauvegarde de l'ancien index

Le dossier **old_index** contiendra une copie de notre fichier **index.js**. Ce dossier servira à conserver les tests effectués avant la refonte du projet.

■ Nettoyage du fichier principal

Le fichier **index.js**, situé à la racine du projet, doit être vidé pour repartir sur une base propre et bien structurée.

Maintenant tout est en ordre nous allons installer **prisma**, une ORM qui va permettre l'interaction avec la base de données. mais nous repellons que vous pouvez utiliser autre ORM de votre choix telques **Postgraphile** **Hasura**

Passons donc à la configuration de prisma.

12 Configuration de Prisma en ligne de commande

— Installation de Prisma et de son client

```
1 npm install prisma --save-dev
2 npm install @prisma/client
3
```

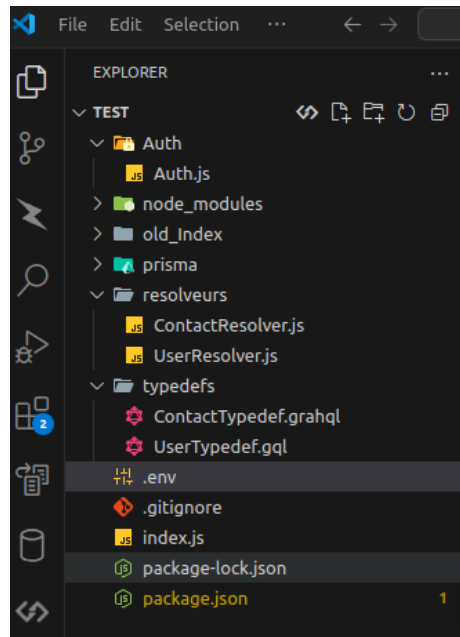
— Initialisation de Prisma

```
1 npx prisma init
2
```

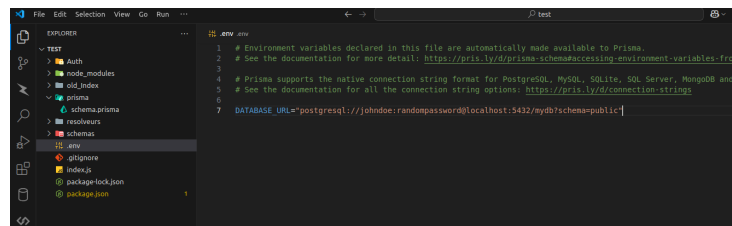
Si vous exécutez la commande **npx prisma init**, un dossier **prisma** apparaîtra, contenant notamment un fichier **.env**. Ce fichier de configuration vous permettra de définir les paramètres de votre base de données, ainsi que différentes variables d'environnement.

En accédant au répertoire **prisma**, vous y trouverez un fichier **schema.prisma**. C'est dans ce fichier que nous allons définir le schéma de la base de données.

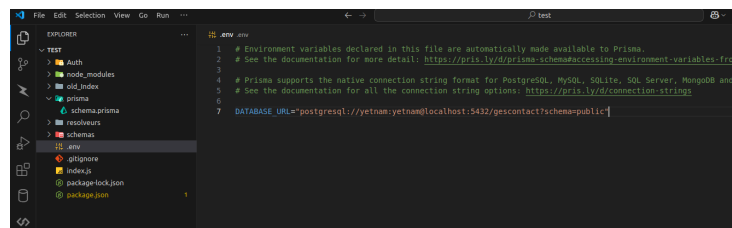
Voici à quoi devrait ressembler notre aborescence à présent.



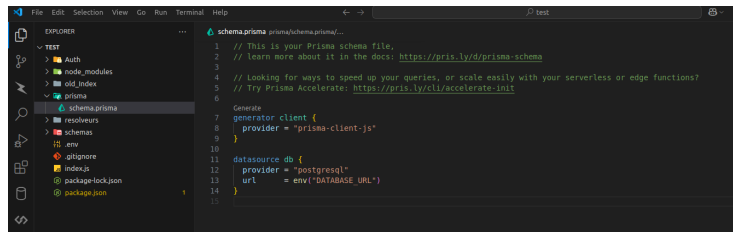
Nous allons accéder au fichier **.env** afin d'effectuer les configurations nécessaires. À la base, voici à quoi ressemble le contenu du fichier **.env**



Vous devez remplacer ces données par les informations correctes, à savoir le nom d'utilisateur, le mot de passe et le nom de la base de données. Je rappelle que dans notre cas, nous utilisons **PostgreSQL**, mais vous pouvez opter pour **SQLite**, **MySQL**, **SQL Server**, **MongoDB**, **CockroachDB**, etc., selon votre choix. Voici comment nous avons effectué notre configuration.



Accédons au fichier **schema.prisma** et définissons notre schéma, à savoir **User** et **Contact**. Mais avant cela, voici l'architecture de base de Prisma.



Voici la manière dont nous avons défini la structure de notre table.

```

1 // This is your Prisma schema file,
2 // learn more about it in the docs: https://pris.ly/d/prisma-schema
3
4 // Looking for ways to speed up your queries, or scale easily with
5 // your serverless or edge functions?
6 // Try Prisma Accelerate: https://pris.ly/cli/accelerate-init
7
8 generator client {
9   provider = "prisma-client-js"
10 }
11
12 datasource db {
13   provider = "postgresql"
14   url      = env("DATABASE_URL")
15 }
16
17 model User {
18   id          Int           @id @default(autoincrement())
19   name        String
20   email       String        @unique
21   password    String
22   contacts    Contact[]
23   createdAt   DateTime      @default(now())
24   updatedAt   DateTime      @updatedAt
25 }
26
27 model Contact {
28   id          Int           @id @default(autoincrement())
29   firstName   String
30   lastName    String
31   phone       String
32   email       String        @unique
33   address     String?
34   userId      Int
35   user        User          @relation(fields: [userId], references: [id])
36   createdAt   DateTime      @default(now())
37   updatedAt   DateTime      @updatedAt
38 }

```

Ensuite, exécutez la commande suivante pour créer la base de données et générer le client Prisma :

Création + application de migration

```
1 npx prisma migrate dev --name init
```

2



Génération de Prisma Client

1

```
npx prisma generate
```

2

Après ce la tapez cette commande pour explorer la base de donnees et ajouter les donner directement dans l'interface qu'offre prisma.

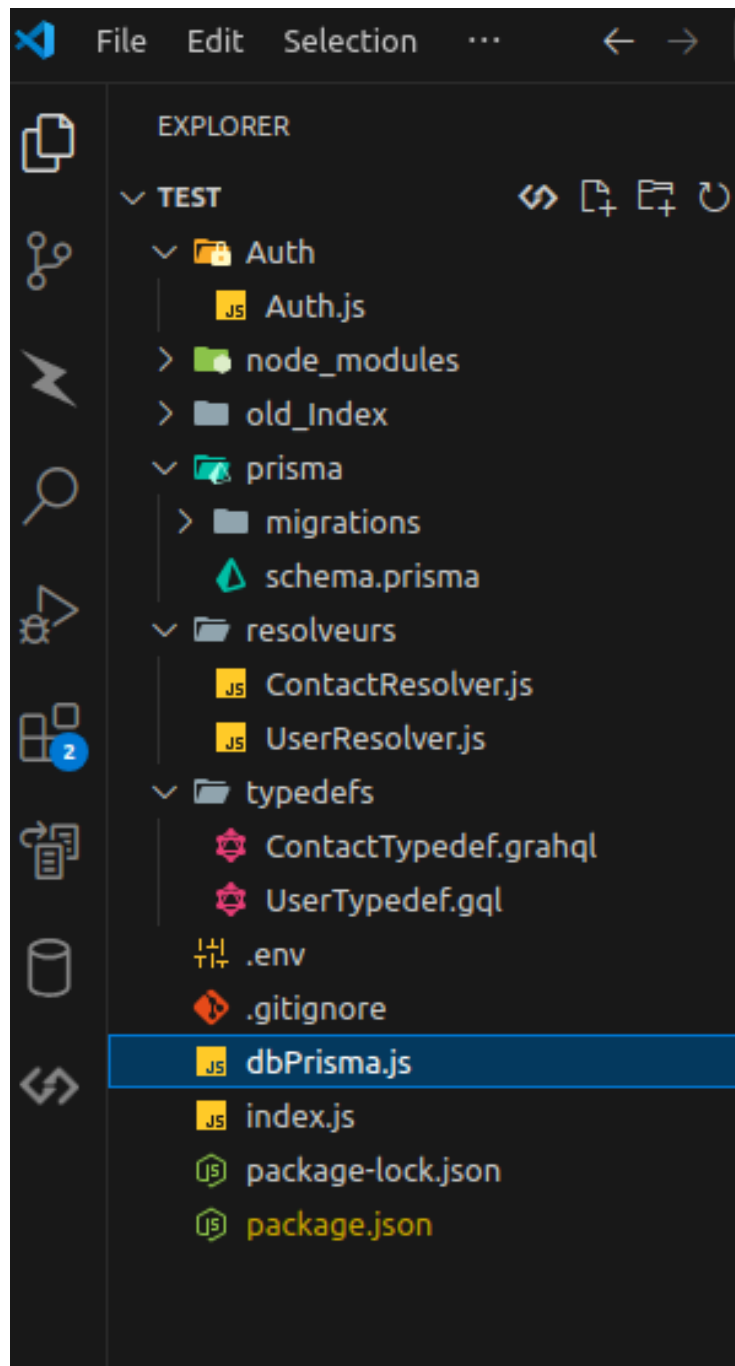
Exploration de la base de données

1

```
npx prisma studio
```

2

Ou vous pouvez egalemnt créer un fichier **dbPrisma.js** a la racine et met ce script à l'interieur pour nous generer quelques seeders pour nos differents tests.



```
1 import { PrismaClient } from '@prisma/client';  
2
```

```

3 const prisma = new PrismaClient();
4 async function main() {
5   const user = await prisma.user.create({
6     data: {
7       name: 'John Doe',
8       email: 'johndoe@example.com',
9       password: 'securepassword123',
10      contacts: {
11        create: [
12          {
13            firstName: 'Alice',
14            lastName: 'Smith',
15            phone: '123-456-7890',
16            email: 'alice.smith@example.com',
17            address: '123 Main St, Springfield',
18          },
19          {
20            firstName: 'Bob',
21            lastName: 'Johnson',
22            phone: '987-654-3210',
23            email: 'bob.johnson@example.com',
24            address: '456 Elm St, Springfield',
25          }
26        ]
27      }
28    }
29  });
30
31  console.log('Utilisateur cree :', user);
32 }
33
34 main()
35   .catch(e => {
36     throw e;
37   })
38   .finally(async () => {
39     await prisma.$disconnect();
40   });

```

Après ce la tapez cette commande pour executer les données de test

Insertion des données

```

1 node dbPrisma.js
2

```

Passons maintenant aux choses sérieuses. Comme cela a été mentionné à plusieurs reprises, en GraphQL, il est primordial de commencer par déclarer les types avant de se lancer dans la logique de l'API. Cette étape fondamentale permet de définir clairement la structure des données et de garantir que toutes les requêtes et mutations respectent un schéma précis. Une fois les types définis, nous pourrons ensuite aborder les fonctionnalités de l'API et la logique des résolveurs qui manipulent ces types.

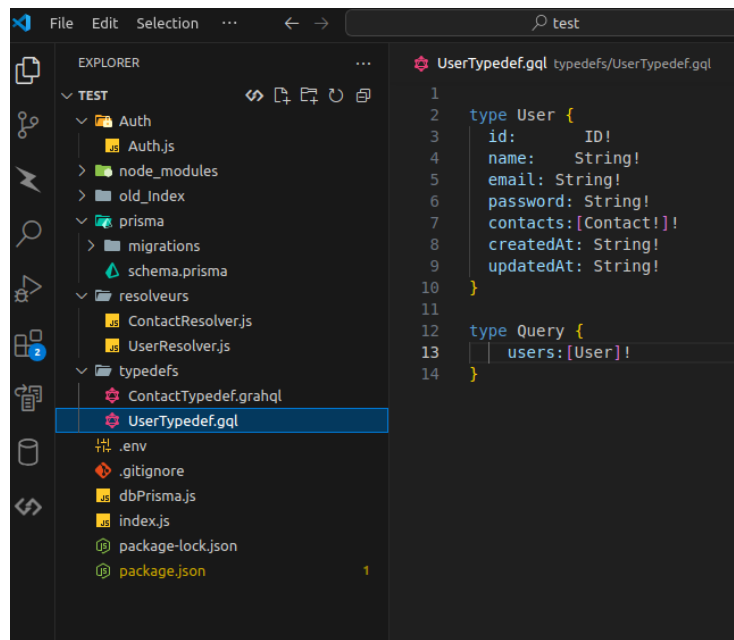
Définissons donc notre schema.

Allez donc dans le fichier `schema/UserTypedef.graphql`

```

1
2 type User {
3   id: ID!
4   name: String!
5   email: String!
6   password: String!
7   contacts:[Contact!]!
8   createdAt: String!
9   updatedAt: String!
10 }
11
12 type Query {
13   users:[User]!
14 }
15
16

```



et dans le fichier **ContactTypedef.gql**

```

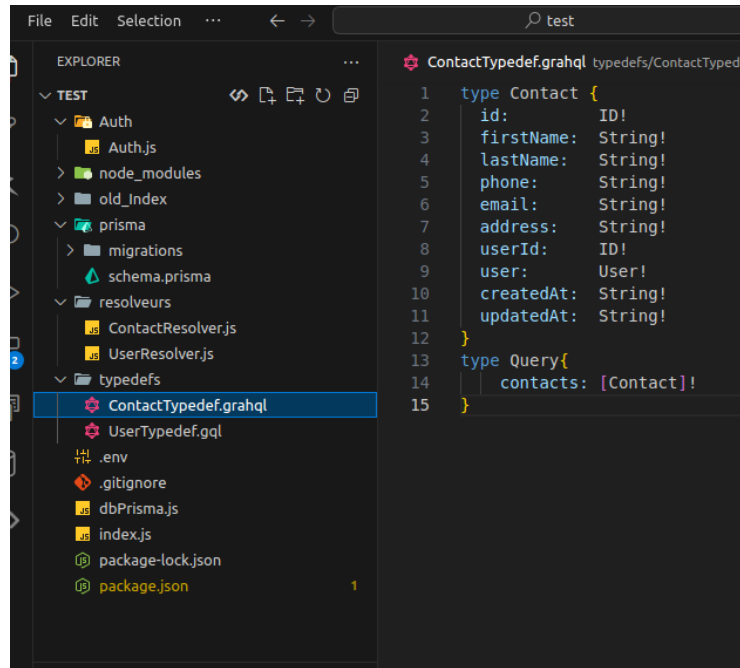
1 type Contact {
2   id: ID!
3   firstName: String!
4   lastName: String!
5   phone: String!
6   email: String!
7   address: String!
8   userId: ID!
9   user: User!
10  createdAt: String!
11  updatedAt: String!
12 }

```

```

13 type Query{
14   contacts: [Contact]!
15 }
16 }
17

```



Maintenant que nous fini avec les **typedef** , allons dans le resolvers. Il faut vraiment noter que graphql est vraiment exigeant sur les types sans c'est type , il nous serait impossible de demarrer notre serveur , donc il faut declarer les types meme si nous n'allons pas encore dfinir le corps dans le resolvers. Accedons à notre fichier, **resolvers/UserResolver.js**, ceci fait nous allons juste declarer le resolveur qui affiche la liste de tous les tutilisateurs. Voici le contenu du fichier **UserResolver.js**

```

1 import { PrismaClient } from "@prisma/client";
2 const prisma = new PrismaClient();
3 const userResolvers={
4   Query: {
5     users: async () =>{},
6   },
7 };
8 export default userResolvers;
9

```

Voici pour contact egalement.

```

1 import { PrismaClient } from "@prisma/client";
2 const prisma = new PrismaClient();

```

```

3 const userResolvers={
4   Query: {
5     users: async () =>{},
6   },
7 };
8 export default userResolvers;
9

```

Nous espérons que vous vous souvenez de la petite histoire de **gql** et **graphql**, un langage de requête révolutionnaire pour les API! ☺
Maintenant que nous avons fini avec nos schemas.

```

1 import { ApolloServer } from "@apollo/server"
2 import { startStandaloneServer } from "@apollo/server/standalone"
3 import gql from "graphql-tag"
4 import fs from 'fs'
5 import userResolvers from './resolvers/UserResolver.js'
6 import contactResolvers from './resolvers/ContactResolver.js'
7
8 const typeDefs = gql(`
9   ${fs.readFileSync('typedefs/UserTypedef.gql','utf8')}
10  ${fs.readFileSync('typedefs/ContactTypedef.graphql','utf8')}
11  `)
12 const resolvers = {
13   Query:{
14     ...userResolvers.Query,
15     ...contactResolvers.Query
16   },
17 }
18 const server = new ApolloServer(
19   { typeDefs, resolvers }
20 )
21 startStandaloneServer(server,{
22   listen:4000, path:'/graphql'}).then(({url})=>{
23   console.log('serveur demare sur ${url}');
24
25 })
26
27

```


Structure du code

1. Importation :

```
1 import { ApolloServer } from "@apollo/server"
2 import { startStandaloneServer } from "@apollo/server/standalone"
3 import gql from "graphql-tag"
4 import fs from 'fs'
5 import userResolvers from './resolvers/UserResolver.js'
6 import contactResolvers from './resolvers/ContactResolver.js'
```

Ici nous faisons les importations ,

ligne 1 Cette ligne importe le ApolloServer à partir du package @apollo/server. ApolloServer est une bibliothèque qui permet de configurer et de lancer un serveur GraphQL. Il est utilisé pour définir le schéma, les résolveurs, et exécuter les requêtes des clients.

ligne 2 Cette ligne importe la fonction startStandaloneServer à partir du module @apollo/server/standalone. .

ligne 3 Cette ligne importe gql de la bibliothèque graphql-tag. graphql-tag est un utilitaire qui permet d'annoter les requêtes GraphQL en JavaScript.

ligne 4

Cette ligne importe le module fs de Node.js. fs est un module natif de Node.js qui permet de manipuler le système de fichiers (par exemple, lire, écrire, supprimer des fichiers). Cela peut être utilisé pour charger des fichiers comme des schémas GraphQL définis dans un fichier .graphql ou .gql.

ligne 5

Cette ligne importe les resolvers pour le type User depuis un fichier externe UserResolver.js. Les résolveurs sont des fonctions qui spécifient la logique d'exécution pour les différentes opérations de GraphQL (comme la récupération de données).

ligne 6 Cette ligne fait la même chose que la ligne 4 mais cette fois-ci c'est pour la table **Contact**

</> Explication du contenu de `index.js`

2. Lecture des typedef :

```
1 const typeDefs = gql(`
2   ${fs.readFileSync('typedefs/UserTypedef.gql', 'utf8')}
3   ${fs.readFileSync('typedefs/ContactTypedef.graphql', 'utf8')}
4 `)
```

Ici nous permettons la lecture des type definis afin de le rendre accessible dans le fichier `index.js`, puisque qu'il le faut pour le demarrage du serveur

3. Lecture des typedef :

```
1 const resolvers = {
2   Query:{
3     ...userResolvers.Query,
4     ...contactResolvers.Query
5   },
6 }
```

`...userResolvers.Query` et `...contactResolvers.Query` : Ces deux lignes utilisent l'opérateur spread pour prendre tous les résolveurs (fonctions de lecture) définis dans `userResolvers.Query` et `contactResolvers.Query`, et les "fusionner" dans un seul objet.

Ce code est une façon de combiner plusieurs ensembles de résolveurs pour les rendre accessibles dans un seul objet `Query`.

4. Instanciation de `ApolloServer` avec les argument `typeDef` et `reolvers` :

```
1 const server = new ApolloServer(
2   { typeDefs, resolvers }
3 )
```

5. Instanciation de `ApolloServer` avec les argument `typeDef` et `reolvers` :

```
1 const server = new ApolloServer(
2   { typeDefs, resolvers }
3 )
```

Ce code démarre un serveur Apollo qui écoute sur le port 4000 et expose l'API GraphQL à l'URL `/graphql`. Une fois le serveur démarré, il affiche l'URL dans la console pour indiquer où l'API est accessible.

Enfin si vous avez suivi correctement toutes ces etapes et que vous retapez la commande **npm run dev** le serveur va demarrer correctement.

À présent, nous allons afficher la liste des utilisateurs et des contacts depuis la base de données. Si vous avez bien suivi, vous avez probablement remarqué

que nous avons défini, au niveau du typedef, la requête qui permet d'afficher l'ensemble des utilisateurs et des contacts. Il ne nous reste plus qu'à définir la logique dans le résolveur afin d'afficher les listes correspondantes de manière dynamique.

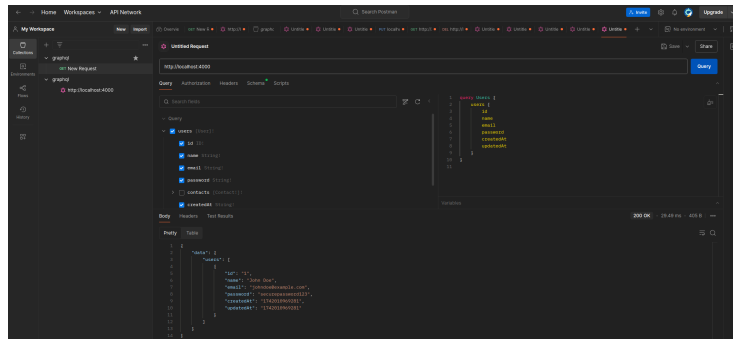
Voici le contenu du fichier **UserResolver.js**

```
1 const prisma = new PrismaClient();
2 const userResolvers={
3   Query: {
4     users: async () =>{
5       return await prisma.user.findMany
6       (
7         { include: { contacts: true } }
8       )
9     },
10  },
11 };
12 export default userResolvers;
```

et voici pour **ContactResolver**

```
1
2 import { PrismaClient } from "@prisma/client";
3 const contactResolvers = {
4   Query: {
5     contacts: async () => await Prisma.contact.findMany({
6       include: { user: true } }),
7   },
8 };
9 export default contactResolvers;
```

Nous pouvons aller dans postman pour faire le test.



Vous pouvez également tester avec les contacts. Vous remarquerez que Postman permet d'afficher les contacts associés à un utilisateur, ainsi que l'utilisateur auquel chaque contact est rattaché. Cela permet une navigation fluide entre les utilisateurs et leurs contacts, facilitant ainsi l'exploitation des relations entre ces entités.

Maintenant nous allons afficher un contact spécifique, comme d'habitude, nous

allons definir la requette d'abord puis apres le resolveur.

Voici la maniere dont nous avons définit la requette .

```
1 contact(id: ID!): Contact!
```

Le fichier **typedefs/ContactTypedef.graphql** devrait ressembler à ceci :

```
1 type Contact {
2   id: ID!
3   firstName: String!
4   lastName: String!
5   phone: String!
6   email: String!
7   address: String!
8   userId: ID!
9   user: User!
10  createdAt: String!
11  updatedAt: String!
12 }
13 type Query{
14   contacts: [Contact]!
15   contact(id: ID!): Contact!
16 }
```

etn voici le resolveur :

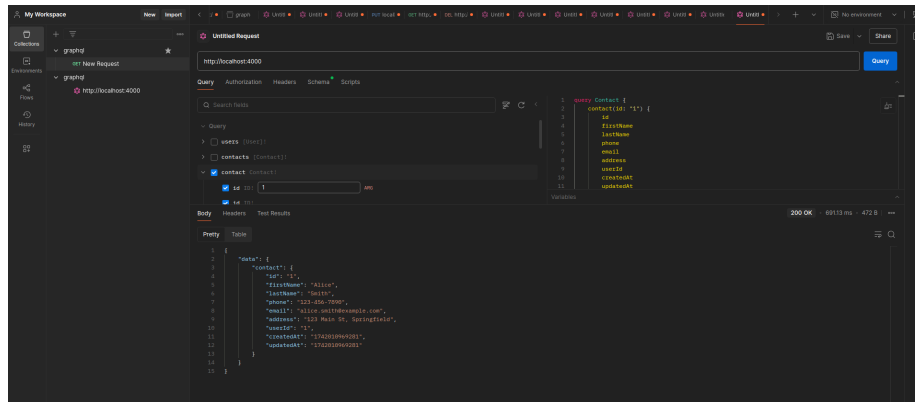
```
1 contact: async(_, {id})=> await prisma.contact.findUnique({
2   where: {id: parseInt(id)},
3   include: {user:true},
4   })
```

Le fichier **resolveurs/ContactResolver.js** devrait ressembler à ceci :

```
1 import { PrismaClient } from "@prisma/client";
2
3 const prisma = new PrismaClient();
4
5 const contactResolvers = {
6   Query: {
7     contacts: async () => await prisma.contact.findMany({
8       include: { user: true } }),
9     contact: async(_, {id})=> await prisma.contact.findUnique({
10      where: {id: parseInt(id)},
11      include: {user:true},
12    })
13   },
14   Mutation:{
15
16   },
17 };
18 };
19
20 export default contactResolvers;
```

Si vous retournez dans Postman et actualisez l'interface, vous verrez qu'une nouvelle requête a été ajoutée. En cliquant dessus, un champ apparaîtra, vous

permettant de saisir l’ID de l’utilisateur que vous souhaitez afficher. Après avoir inséré l’ID, les informations correspondantes s’afficheront.



13 Nous allons aborder le concept de mutation (create, update, delete)

, toujours dans le fichier `typedefs/ContactTypedef.graphql`, ajoutez donc ce nouveau code.

```
1
2 type Mutation {
3   ajouterContact(firstName: String!, lastName:String,phone:String,
4     email:String, address:String, userId:String):Contact
5 }
```

Ce fichier doit maintenant ressembler a ceci

```
1 type Contact {
2   id: ID!
3   firstName: String!
4   lastName: String!
5   phone: String!
6   email: String!
7   address: String!
8   userId: ID!
9   user: User!
10  createdAt: String!
11  updatedAt: String!
12 }
13
14 type Query{
15   contacts: [Contact]!
16   contact(id: ID!): Contact!
17 }
18
19 type Mutation {
20   ajouterContact(firstName: String!, lastName:String,phone:String,
21     email:String, address:String, userId:String):Contact
22 }
```

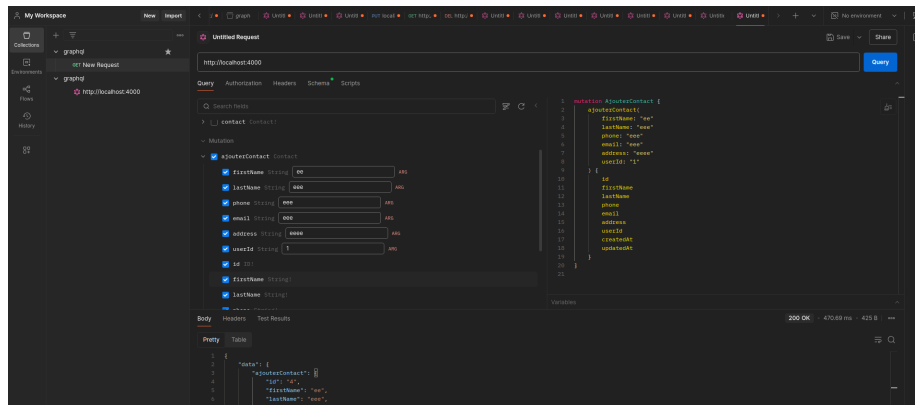
Si nous finissons, maintenant définissons le resolveur , voici le code.

```
1  Mutation: {
2    ajouterContact: async (_, { firstName, lastName, phone,
3      email, address, userId }) => {
4      const nouvelContact = await prisma.contact.create({
5        data: {
6          firstName,
7          lastName,
8          phone,
9          email,
10         address,
11         userId: parseInt(userId, 10)
12       }
13     });
14     return nouvelContact;
15   }
16 }
17 };
```

Le fichier devrait ressembler à ceci

```
1  import { PrismaClient } from "@prisma/client";
2
3  const prisma = new PrismaClient();
4  const contactResolvers = {
5    Query: {
6      contacts: async () => await prisma.contact.findMany({
7        include: { user: true } }),
8      contact: async (_, {id})=> await prisma.contact.findUnique({
9        where: {id: parseInt(id)},
10       include: {user:true},
11     })
12   },
13   Mutation: {
14     ajouterContact: async (_, { firstName, lastName, phone,
15       email, address, userId }) => {
16       const nouvelContact = await prisma.contact.create({
17         data: {
18           firstName,
19           lastName,
20           phone,
21           email,
22           address,
23           userId: parseInt(userId, 10)
24         }
25       });
26       return nouvelContact;
27     }
28   }
29 };
30
31 export default contactResolvers;
```

Voici un exemple de test.



Faisons la mise a jour

```

1 import { PrismaClient } from "@prisma/client";
2
3 const prisma = new PrismaClient();
4 const contactResolvers = {
5   Query: {
6     contacts: async () => await prisma.contact.findMany({
7       include: { user: true } }),
8     contact: async (_, {id})=> await prisma.contact.findUnique({
9       where: {id: parseInt(id)},
10      include: {user:true},
11    })
12  },
13  Mutation: {
14    ajouterContact: async (_, { firstName, lastName, phone,
15      email, address, userId }) => {
16      const nouvelContact = await prisma.contact.create({
17        data: {
18          firstName,
19          lastName,
20          phone,
21          email,
22          address,
23          userId: parseInt(userId, 10)
24        }
25      });
26      return nouvelContact;
27    }
28  }
29 };
30
31 export default contactResolvers;

```

Voici la definition de mise a jour

```

1 modifierContact(id:ID, firstName: String, lastName:String,phone:
  String, email:String, address:String, userId:String):Contact

```

Maintenant voici le resolveur

```
1
2  modifierContact: async (_, { id, firstName, lastName, phone,
3    email, address }) => {
4      const contactIdInt = parseInt(id, 10);
5
6      if (isNaN(contactIdInt)) {
7        throw new Error("L'ID doit tre un nombre valide.");
8      }
9
10     try {
11       const updatedContact = await prisma.contact.update({
12         where: {
13           id: contactIdInt,
14         },
15         data: {
16           firstName: firstName || undefined,
17           lastName: lastName || undefined,
18           phone: phone || undefined,
19           email: email || undefined,
20           address: address || undefined,
21         },
22       });
23       return updatedContact;
24     } catch (error) {
25       throw new Error("Erreur lors de la mise jour du
26       contact : " + error.message);
27     }
28   },
```

Voici le contenu de tout le fichier jusqu'a present

```
1
2  import { PrismaClient } from "@prisma/client";
3
4  const prisma = new PrismaClient();
5  const contactResolvers = {
6    Query: {
7      contacts: async () => await prisma.contact.findMany({
8        include: { user: true } }),
9      contact: async (_, {id})=> await prisma.contact.findUnique({
10        where: {id: parseInt(id)},
11        include: {user:true},
12      })
13    },
14    Mutation: {
15      ajouterContact: async (_, { firstName, lastName, phone,
16        email, address, userId }) => {
17        const nouvoContact = await prisma.contact.create({
18          data: {
19            firstName,
20            lastName,
21            phone,
22            email,
23            address,
24            userId: parseInt(userId, 10)
25          }
26        })
27      }
28    }
29  }
```



```

24     }
25   });
26
27   return nouvoContact;
28 },
29
30   modifierContact: async (_, { id, firstName, lastName, phone
31   , email, address }) => {
32     const contactIdInt = parseInt(id, 10);
33
34     if (isNaN(contactIdInt)) {
35       throw new Error("L'ID doit tre un nombre valide.");
36     }
37
38     try {
39       const updatedContact = await prisma.contact.update({
40         where: {
41           id: contactIdInt,
42         },
43         data: {
44           firstName: firstName || undefined,
45           lastName: lastName || undefined,
46           phone: phone || undefined,
47           email: email || undefined,
48           address: address || undefined,
49         },
50       });
51
52       return updatedContact;
53     } catch (error) {
54       throw new Error("Erreur lors de la mise jour du
55       contact : " + error.message);
56     }
57   },
58 };
59 export default contactResolvers;

```

Vous pouvez faire les tests a votre guise.

Suppression de contact

Pour supprimer un contact , c'est toujours la meme procedure : on definit le schema la mutation comme telle

```

1   supprimerContact(id: ID): Boolean
2

```

Voici a quoi devrait maintenant ressembler le fichier

```

1 type Contact {
2   id: ID!
3   firstName: String!
4   lastName: String!
5   phone: String!
6   email: String!
7   address: String!
8   userId: ID!

```

```

9   user:      User!
10  createdAt: String!
11  updatedAt: String!
12 }
13
14 type Query{
15   contacts: [Contact]!
16   contact(id: ID!): Contact!
17 }
18
19 type Mutation {
20   ajouterContact(firstName: String, lastName:String,phone:String,
21     email:String, address:String, userId:String):Contact,
22   modifierContact(id:ID, firstName: String, lastName:String,phone:
23     String, email:String, address:String, userId:String):Contact,
24   supprimerContact(id:ID):Boolean
25 }

```

ensuite on definit le resolveur

```

1  supprimerContact: async (_, { id }) => {
2    const contactIdInt = Number(id);
3
4    if (isNaN(contactIdInt)) {
5      throw new Error("L'ID doit etre un nombre valide.");
6    }
7
8    try {
9      const deletedContact = await prisma.contact.delete({
10        where: {
11          id: contactIdInt,
12        },
13      });
14
15      return true;
16    } catch (error) {
17      throw new Error("Erreur lors de la suppression du contact :
18        " + error.message);
19    }
20  },

```

Après le fichier resolver devrait ressembler a ceci

```

1  import { PrismaClient } from "@prisma/client";
2
3  const prisma = new PrismaClient();
4  import { verifyToken } from '../Auth/AuthUtils.js';
5  const contactResolvers = {
6    Query: {
7
8      contacts: async () => await prisma.contact.findMany({ include:
9        { user: true } }),
10
11      contact: async (_, { id }) =>
12        await prisma.contact.findUnique({

```

```

13       where: { id: parseInt(id) },
14       include: { user: true },
15     }},
16   },
17
18   Mutation: {
19     ajouterContact: async (_, { firstName, lastName, phone, email,
20       address, userId }) => {
21       const nouvelContact = await prisma.contact.create({
22         data: {
23           firstName,
24           lastName,
25           phone,
26           email,
27           address,
28           userId: parseInt(userId, 10),
29         },
30       });
31
32       return nouvelContact;
33     },
34
35     modifierContact: async (_, { id, firstName, lastName, phone,
36       email, address }) => {
37       const contactIdInt = parseInt(id, 10);
38
39       if (isNaN(contactIdInt)) {
40         throw new Error("L'ID doit être un nombre valide.");
41       }
42
43       try {
44         const updatedContact = await prisma.contact.update({
45           where: {
46             id: contactIdInt,
47           },
48           data: {
49             firstName: firstName || undefined,
50             lastName: lastName || undefined,
51             phone: phone || undefined,
52             email: email || undefined,
53             address: address || undefined,
54           },
55         });
56
57         return updatedContact;
58       } catch (error) {
59         throw new Error("Erreur lors de la mise à jour du contact : "
60           + error.message);
61       }
62     },
63
64     supprimerContact: async (_, { id }) => {
65       const contactIdInt = Number(id);
66
67       if (isNaN(contactIdInt)) {
68         throw new Error("L'ID doit être un nombre valide.");
69       }

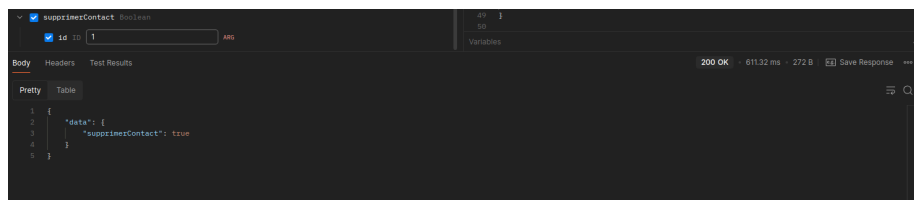
```

```

67
68     try {
69         const deletedContact = await prisma.contact.delete({
70             where: {
71                 id: contactIdInt,
72             },
73         });
74
75         return true;
76     } catch (error) {
77         throw new Error("Erreur lors de la suppression du contact : "
78             + error.message);
79     }
80 },
81 },
82 };
83
84 export default contactResolvers;
85
86

```

Voici le resultat :



14 Implémentation de l'authentification avec JWT et BCryptJS

Pour implémenter un système d'authentification en utilisant GraphQL, voici le processus étape par étape :

14.1 Définition du Schéma GraphQL

Dans un premier temps, nous devons définir deux mutations dans notre schéma GraphQL :

- **Mutation register** : Cette mutation permet à un utilisateur de s'enregistrer en fournissant un nom , un email et un mot de passe.
- **Mutation login** : Cette mutation permet à un utilisateur de se connecter avec son email et son mot de passe.

La mutation **login** retournera un objet contenant un **token JWT** et les **informations de l'utilisateur**.

14.2 Modèle User

Le modèle **User** contient déjà les champs suivants : **name**, **email** et **password**. Nous allons **hacher** le mot de passe avant de le stocker dans la base de données. Pour ce faire, nous utiliserons un module comme **bcryptjs** afin de sécuriser les mots de passe des utilisateurs.

14.3 Processus de la Mutation register

- Lors de l'inscription, la mutation **register** devra effectuer plusieurs étapes :
- Vérifier si un utilisateur avec le même **email**
 - Si l'utilisateur n'existe pas, un nouvel utilisateur sera créé avec les informations fournies.
 - Le mot de passe de l'utilisateur sera **haché** à l'aide de **bcryptjs**.
 - Un **token JWT** sera généré pour l'utilisateur, afin qu'il puisse être utilisé dans les requêtes futures.
 - Le **token** et les informations de l'utilisateur (par exemple **id**, **email**) seront retournées en réponse.

14.4 Processus de la Mutation login

- Lors de la connexion, la mutation **login** devra suivre ces étapes :
- Vérifier si l'utilisateur existe en recherchant par **email**.
 - Comparer le mot de passe fourni par l'utilisateur avec le mot de passe haché stocké dans la base de données en utilisant **bcryptjs**.
 - Si les mots de passe correspondent, un **token JWT** sera généré pour l'utilisateur et retourné.
 - En cas d'échec de l'authentification (mauvais mot de passe), une erreur sera renvoyée.

14.5 Gestion du Token JWT

Le token JWT généré lors de l'inscription ou de la connexion contient les informations de l'utilisateur (par exemple, **id**) et une date d'expiration. Ce **token JWT** est envoyé au client et doit être inclus dans les en-têtes des requêtes futures afin d'accéder aux ressources protégées. Le token sera généralement placé dans l'en-tête **Authorization** des requêtes.

14.6 Sécurisation des Résolveurs

Afin de protéger certaines routes de l'application, un middleware sera utilisé pour vérifier la validité du token JWT. Cela garantit que seules les requêtes contenant un token valide sont autorisées à accéder aux données sensibles ou à effectuer certaines actions sur le serveur.

14.7 Résumé du Processus

En résumé, voici les principales étapes de l'implémentation de l'authentification :

- Création des mutations **register** et **login** dans le schéma GraphQL.
- Hachage des mots de passe avec **bcryptjs**.
- Génération d'un **token JWT** lors de l'inscription et de la connexion.
- Vérification du token JWT dans les requêtes futures pour sécuriser les ressources.

Ainsi, ce processus permet de mettre en place une authentification sécurisée dans une application GraphQL en utilisant les mutations **register** et **login**, avec un système de gestion des tokens JWT pour l'accès aux ressources protégées.

Maintenant que tout est bien expliqué et détaillé, nous allons commencer par manipuler cela :

Pour se faire, nous allons d'abord l'installer avec la commande :

```
1 npm install jsonwebtoken
```

Cryptage du mot de passe pour des raisons de sécurité

```
1 npm install bcryptjs
```

Comme dit, allons créer les mutations **login** et **register** mais avant tout définissons le type personnalisé **Register** et **Login** pour gérer la réponse d'authentification après une opération de connexion ou d'inscription.

```
1
2 type Register {
3   id: ID!
4   name: String!
5   email: String!
6   password: String!
7   createdAt: String!
8   updatedAt: String!
9 }
10 type Login {
11   token: String!
12   user: User!
13 }
14
15 type Mutation {
16   register(name: String!, email: String!, password: String!):
     Register!
17   login(email: String!, password: String!): Login!
18 }
19
```

Le fichier **typedefs/UserTypedef.gql** devrait ressembler à ceci :

```
1
2
```

```

3
4 type User {
5   id: ID!
6   name: String!
7   email: String!
8   password: String!
9   contacts:[Contact]!
10  createdAt: String!
11  updatedAt: String!
12 }
13
14
15 type Register {
16   id: ID!
17   name: String!
18   email: String!
19   password: String!
20   createdAt: String!
21   updatedAt: String!
22 }
23
24 type Query {
25   users:[User]!
26 }
27
28 type Login {
29   token: String!
30   user: User!
31 }
32
33 type Mutation {
34   register(name: String!, email: String!, password: String!):
     Register!
35   login(email: String!, password: String!): Login!
36 }
37
38
39

```

Maintenant que cela est fait, nous allons nous attaquer au dossier **Auth**, que nous n'avons pas encore édité jusqu'à présent. Bien que ce ne soit pas absolument nécessaire, pour une meilleure organisation et cohérence, il est conseillé de renommer la lettre **A**, qui était en majuscule, en **a** afin de correspondre aux autres noms de dossiers.

Nous allons d'abord créer un utilisateur en hachant le mot de passe.

Voici le contenu de **Auth.js**

```

1 import bcrypt from "bcryptjs";
2 import jwt from "jsonwebtoken";
3
4
5 // Hashage du mot de passe
6 export const hashPassword = async (password) => {

```

```

7   return await bcrypt.hash(password, 10);
8 };
9
10

```

Dans le fichier **resolvers/UserResolvers.js** voici comment nous avons défini la mutation de l'inscription

```

1  Mutation: {
2    register: async (_, { name, email, password }) => {
3      const existingUser = await prisma.user.findUnique({ where: {
4        email } });
5      if (existingUser) throw new Error("Email deja utilise !");
6
7      const hashedPassword = await hashPassword(password);
8
9      const user = await prisma.user.create({
10        data: {
11          name,
12          email,
13          password: hashedPassword
14        }
15      });
16      return user;
17    },
18  }

```

Le fichier **resolvers/UserResolvers.js** apres ajout du code d'inscription devrait ressembler à ceci :

```

1  import { PrismaClient } from "@prisma/client";
2  import { hashPassword, verifyPassword, generateToken } from "../
3    auth/Auth.js";
4  const prisma = new PrismaClient();
5
6  const userResolvers = {
7    Query: {
8      users: async () => {
9        return await prisma.user.findMany({
10          include: { contacts: true }
11        });
12      },
13    },
14
15    Mutation: {
16      register: async (_, { name, email, password }) => {
17        const existingUser = await prisma.user.findUnique({ where: {
18          email } });
19        if (existingUser) throw new Error("Email deja utilise !");
20
21        const hashedPassword = await hashPassword(password);
22
23        const user = await prisma.user.create({
24          data: {
25            name,

```



```

26         email,
27         password: hashedPassword
28     }
29 });
30
31     return user;
32 },
33
34 };
35 export default userResolvers;
36

```

Après la définition du resolvers, nous allons ajouter la ligne de la mutation de user **contactResolvers.Mutation** aussi dedans .

Voici le contenu de **index.js** :

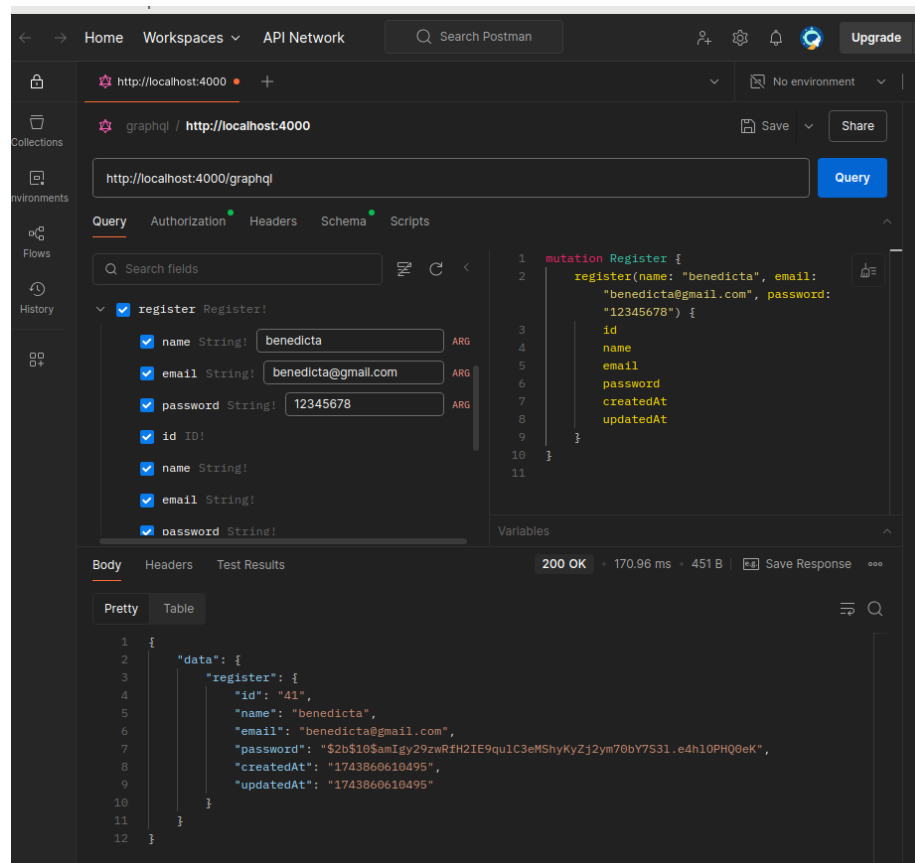
```

1
2
3 import { ApolloServer } from "@apollo/server";
4 import { startStandaloneServer } from "@apollo/server/standalone";
5 import gql from "graphql-tag";
6 import fs from 'fs';
7 import userResolvers from './resolvers/UserResolver.js';
8 import contactResolvers from './resolvers/ContactResolver.js';
9
10 // Charger les typedefs a partir des fichiers externes
11 const typeDefs = gql(`
12   ${fs.readFileSync('typedefs/UserTypedef.gql', 'utf8')}
13   ${fs.readFileSync('typedefs/ContactTypedef.graphql', 'utf8')}
14 `);
15
16 // Combine les resolveurs de contacts et utilisateurs
17 const resolvers = {
18   Query: {
19     ...userResolvers.Query,
20     ...contactResolvers.Query,
21   },
22   Mutation: {
23     ...userResolvers.Mutation,
24     ...contactResolvers.Mutation,
25   },
26 };
27
28 const server = new ApolloServer({
29   typeDefs,
30   resolvers,
31 });
32
33
34
35 startStandaloneServer(server, {
36   listen: { port: 4000 },
37
38 }).then(({ url }) => {
39   console.log('Serveur d marr sur ${url}');
40 });
41
42

```

Pour le test de register, rien n'a changé, allons donc dans postman pour tester l'inscription de l'utilisateur.

Voici ce à quoi vous devez vous attendre.



Maintenant que l'inscription est fait et que nous sommes sur que cela marche allons donc faire la **connexion**.

Allons installer **dotenv**

Est une librairie qui va permettre le chargement des variables d'environnement se trouvant dans le fichier .env

```
1 npm install dotenv
```

allons dans le fichier **auth/Auth.js** pour définir les methodes dont nous aurons besoins pour la connexion

```

1 const SECRET_KEY = process.env.JWT_SECRET || "";
2
3
4 // Hashage du mot de passe
5 export const hashPassword = async (password) => {
6   return await bcrypt.hash(password, 10);
7 };
8 // represente le facteur de cout, ou "salt rounds"
9 //c'est le nombre d'iterations utilisees pour generer le
   hash.
10 //Cela signifie que le processus de hachage est effectue 10 fois
11 // (ou plus precisement, le processus utilise 10 tours de
   calculs
12 // intensifs) pour rendre le hash plus lent et donc plus
   difficile "casser" par brute force
13
14 // Verification du mot de passe
15
16 export const verifyPassword = async (password, hashedPassword) => {
17   return await bcrypt.compare(password, hashedPassword);
18 };
19 //bcrypt.hash(password, 10)
20
21
22
23 // Generation du token JWT
24 export const generateToken = (user) => {
25   return jwt.sign({ id: user.id, email: user.email }, SECRET_KEY, {
     expiresIn: "1m" });
26 };
27 //La methode sign() de la bibliotheque jsonwebtoken est utilisee
   pour creer un JWT.
28
29
30 // Verification du token JWT
31 export const verifyToken = (token) => {
32   try {
33     return jwt.verify(token, SECRET_KEY);
34   } catch (error) {
35     throw new Error("Token invalide ou expire");
36   }
37 };
38

```

Si vous ajoutez ces trois méthodes à celle qui était déjà présente dans le fichier, on aura alors un total de quatre méthodes.

allons dans le fichier **resolvers/UserResolver.js** pour definir la mutation login

```

1
2
3   login: async (_, { email, password }) => {
4     const user = await prisma.user.findUnique({ where: { email }
5   });
6     if (!user) throw new Error("Utilisateur non trouve");
7   }
8 }

```

```

6
7     const valid = await verifyPassword(password, user.password);
8     if (!valid) throw new Error("Mot de passe incorrect");
9
10    const token = generateToken(user);
11
12    return { token, user };
13  }
14 }
15

```

le fichier devrait maintenant ressembler a ceci

```

1
2
3 import { PrismaClient } from "@prisma/client";
4 import { hashPassword, verifyPassword, generateToken } from "../
5   auth/Auth.js";
6 const prisma = new PrismaClient();
7
8 const userResolvers = {
9   Query: {
10
11     users: async () => {
12       return await prisma.user.findMany({
13         include: { contacts: true }
14       });
15     },
16   },
17   Mutation: {
18     register: async (_, { name, email, password }) => {
19       const existingUser = await prisma.user.findUnique({ where: {
20         email } });
21       if (existingUser) throw new Error("Email deja utilise !");
22
23       const hashedPassword = await hashPassword(password);
24
25       const user = await prisma.user.create({
26         data: {
27           name,
28           email,
29           password: hashedPassword
30         }
31       });
32       return user;
33     },
34     login: async (_, { email, password }) => {
35       const user = await prisma.user.findUnique({ where: { email }
36     });
37       if (!user) throw new Error("Utilisateur non trouve");
38       const valid = await verifyPassword(password, user.password);
39       if (!valid) throw new Error("Mot de passe incorrect");
40       const token = generateToken(user);
41       return { token, user };
42     }
43   }
44 }
45

```

```

43 }
44 export default userResolvers;
45

```

Dans le fichier `.env` allons mettre notre clé secrete qui va permettre la génération du token

```

1
2
3
4
5 # Environment variables declared in this file are automatically
  # made available to Prisma.
6 # See the documentation for more detail: https://pris.ly/d/prisma-
  # schema#accessing-environment-variables-from-the-schema
7
8 # Prisma supports the native connection string format for
  PostgreSQL, MySQL, SQLite, SQL Server, MongoDB and CockroachDB.
9 # See the documentation for all the connection string options:
  https://pris.ly/d/connection-strings
10
11 DATABASE_URL="postgresql://yetnam:yetnam@localhost:5432/gescontact?
  schema=public"
12
13
14 JWT_SECRET=fzdfghjkl
15

```

```

1
2
3
4 import { ApolloServer } from "@apollo/server";
5 import { startStandaloneServer } from "@apollo/server/standalone";
6 import gql from "graphql-tag";
7 import fs from 'fs';
8 import userResolvers from './resolvers/UserResolver.js';
9 import contactResolvers from './resolvers/ContactResolver.js';
10 import { verifyToken , generateToken} from './auth/AuthUtils.js';
  // Import de la fonction de v rification du token
11
12 import dotenv from 'dotenv';
13 dotenv.config();
14
15 // Charger les typedefs      partir des fichiers externes
16
17 const typeDefs = gql(`
18   ${fs.readFileSync('typedefs/UserTypedef.gql', 'utf8')}
19   ${fs.readFileSync('typedefs/ContactTypedef.graphql', 'utf8')}
20 `);
21
22 // Combine les resolveurs de contacts et utilisateurs
23
24 const resolvers = {
25   Query: {
26     ...userResolvers.Query,
27     ...contactResolvers.Query,
28   },
29   Mutation: {

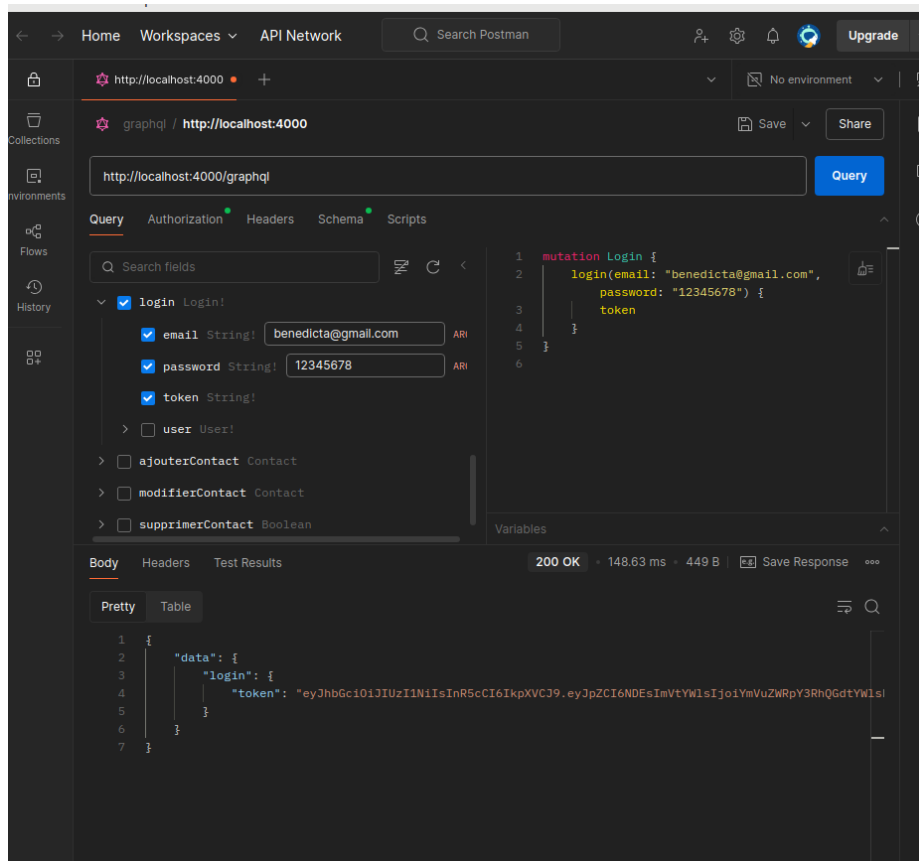
```

```

30     ...userResolvers.Mutation,
31     ...contactResolvers.Mutation,
32   },
33 };
34 const server = new ApolloServer({
35   typeDefs,
36   resolvers,
37 });
38
39 startStandaloneServer(server, {
40   listen: { port: 4000 },
41
42 }).then(({ url }) => {
43   console.log('Serveur démarre sur ${url}');
44 });
45
46

```

maintenant que vous avez chargé la variable d'environnement dans l'index.js
allons tester la connexion avec un utilisateur dont le mot de passe est haché



Maintenant que c'est bon définissons le Middleware pour vérifier le token JWT

```
1
2 const authContext = async ({ req }) => {
3   const token = req.headers.authorization?.replace("Bearer ", "")
4   || "";
5
6   try {
7     const decoded = verifyToken(token);
8
9     // V r i f i c a t i o n   e n   b a s e   d e   d o n n e s
10    const user = await prisma.user.findUnique({
11      where: { id: decoded.id },
12      select: { id: true, name: true, email: true }
13    });
14
15    if (!user) {
16      console.log("Utilisateur invalide");
17      return { user: null, prisma, message: "Utilisateur non trouve"
18        " };
19    }
20
21    console.log("Utilisateur authentifié avec succes !");
22
23    // Retourner l'utilisateur avec les informations
24    // supplémentaires
25    return {
26      user,
27      prisma,
28      message: "Connexion reussie"
29    };
30  } catch (error) {
31    console.log("Erreur de token:", error.message);
32    return { user: null, prisma, message: "Echec de l'
33      authentification" };
34  }
35};
```

On lance le serveur avec le context, qui contient les information de l'utilisateur .

```
1   context: authContext,
2
```

Voici l'index.js

```
1
2 import { PrismaClient } from "@prisma/client";
3 const prisma = new PrismaClient();
4
5 import { ApolloServer } from "@apollo/server";
6 import { startStandaloneServer } from "@apollo/server/standalone";
7 import gql from "graphql-tag";
8 import fs from 'fs';
```

```

9 import userResolvers from './resolveurs/UserResolver.js';
10 import contactResolvers from './resolveurs/ContactResolver.js';
11 import { verifyToken } from './auth/AuthUtils.js';
12 import dotenv from 'dotenv';
13 dotenv.config();
14 // Charger les typeDefs partir des fichiers externes
15 const typeDefs = gql(`
16   ${fs.readFileSync('typeDefs/UserTypedef.gql', 'utf8')}
17   ${fs.readFileSync('typeDefs/ContactTypedef.graphql', 'utf8')}
18 `);
19
20 // Combine les resolveurs de contacts et utilisateurs
21 const resolvers = {
22   Query: {
23     ...userResolvers.Query,
24     ...contactResolvers.Query,
25   },
26   Mutation: {
27     ...userResolvers.Mutation,
28     ...contactResolvers.Mutation,
29   },
30 };
31
32 const authContext = async ({ req }) => {
33   const token = req.headers.authorization?.replace("Bearer ", "")
34   || "";
35   try {
36     const decoded = verifyToken(token);
37
38     // Verification en base de donnees
39     const user = await prisma.user.findUnique({
40       where: { id: decoded.id },
41       select: { id: true, name: true, email: true }
42     });
43
44     if (!user) {
45       console.log("Utilisateur invalide");
46       return { user: null, prisma, message: "Utilisateur non trouve" };
47     }
48
49     console.log("Utilisateur authentifie avec succ s !");
50     // Retourner l'utilisateur avec les informations
51     // suppl mentaires
52     return {
53       user,
54       prisma,
55       message: "Connexion reussie"
56     };
57   } catch (error) {
58     console.log("Erreur de token:", error.message);
59     return { user: null, prisma, message: "chec de l'
60     authentification" };
61   }
62 };
63
64 const server = new ApolloServer({
65   typeDefs,

```



```

62   resolvers,
63 });
64
65 const { url } = await startStandaloneServer(server, {
66   context: authContext,
67   listen: { port: 4001 },
68 });
69 console.log('Serveur démarre sur ${url}graphql');
70
71
72

```

Après avoir fait cela , allons protéger le resolveur de contact

```

1
2   contacts: async (_, args, context) => {
3     console.log("Contexte re u :", context);
4
5     if (!context.user) {
6       console.log("Utilisateur non authentifié :", context.user);
7       throw new Error("Acces refuse !");
8     }
9
10    try {
11      console.log(context.user.id);
12      const contacts = await context.prisma.contact.findMany({
13        where: { userId: context.user.id },
14        include: { user: true }
15      });
16
17      if (!contacts) {
18        throw new Error("Aucun contact trouv pour cet
19        utilisateur.");
20      }
21
22      return contacts;
23    } catch (error) {
24      console.log("Erreur lors de la recuperation des contacts :",
25      , error.message);
26      throw new Error("Erreur interne du serveur.");
27    }
28  },

```

Cette fonction sert à récupérer tous les contacts qui appartiennent à l'utilisateur connecté. Mais elle ne le fait que si l'utilisateur est bien authentifié.

Par défaut le resolveur prend quatre parametre qui sont :

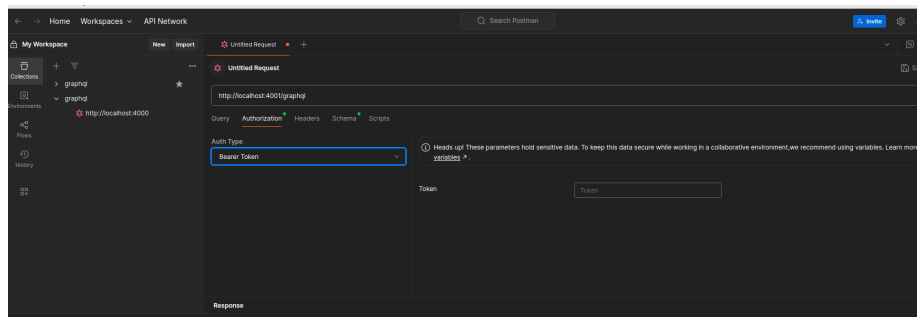
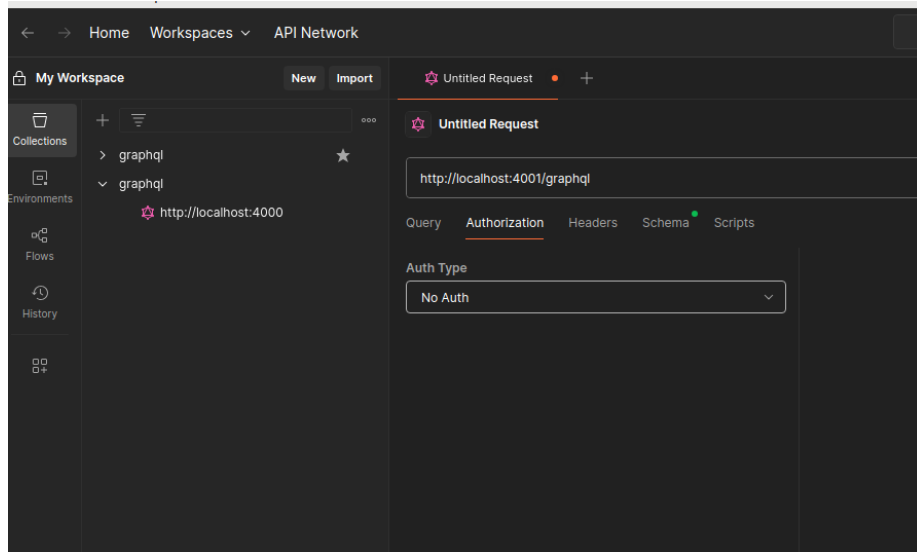
parent : Données du niveau supérieur, utile dans les relations entre objets.

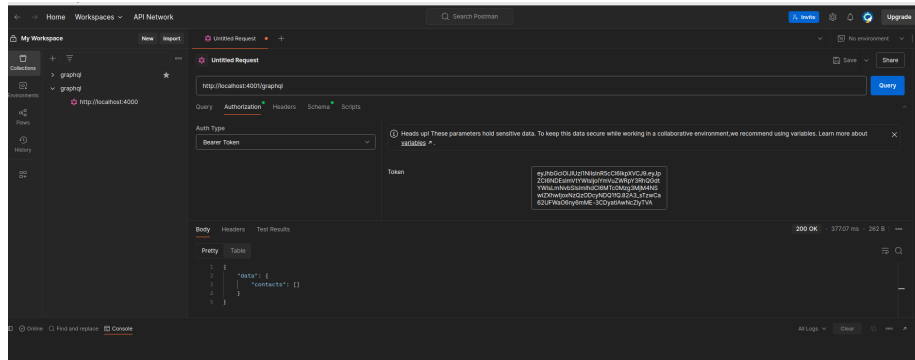
args : Les arguments passés avec la requête ou la mutation (ex. id d'un utilisateur).

context : Données globales partagées, comme l'utilisateur authentifié ou la connexion à la base de données.

info : Détails techniques sur la requête, rarement utilisé mais utile dans des cas avancés.

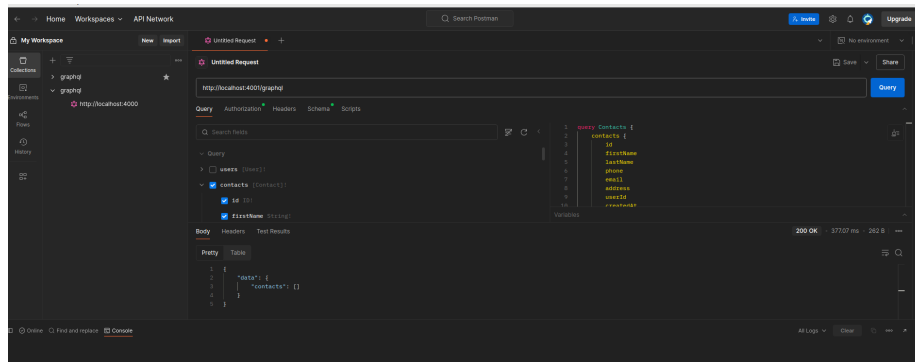
Maintenant pour voir que cela marche, nous allons au niveau de postman, copier le token que nous avons generer lors de l'autentification et mettre dans l'onglet autorisation dans la case ou c'est ecrit token, tou en prenant soin de choisir l'option Bearer Token. voici etape par etape comment proceder.





Une fois le token collé, retournez dans l'onglet Query afin de tester l'affichage des contacts. La requête renverra uniquement les contacts créés par l'utilisateur actuellement connecté.

Dans cet exemple, le résultat sera un tableau vide, car l'utilisateur vient d'être créé et aucun contact ne lui est encore associé.

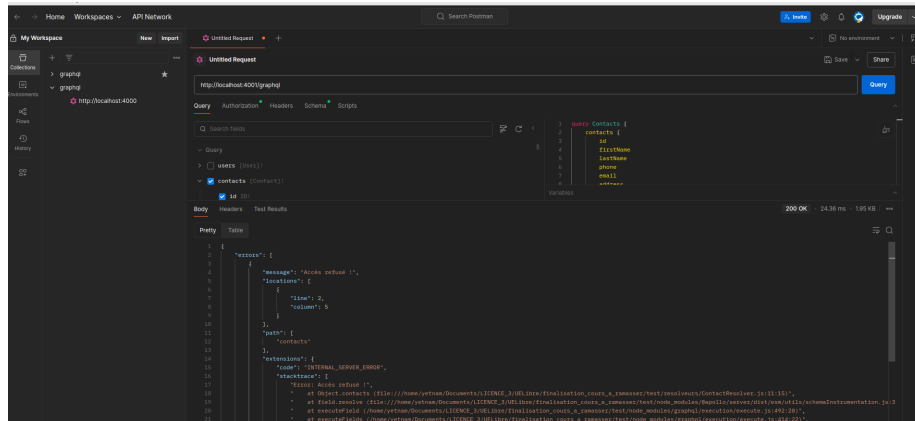


Vérification de l'expiration du token

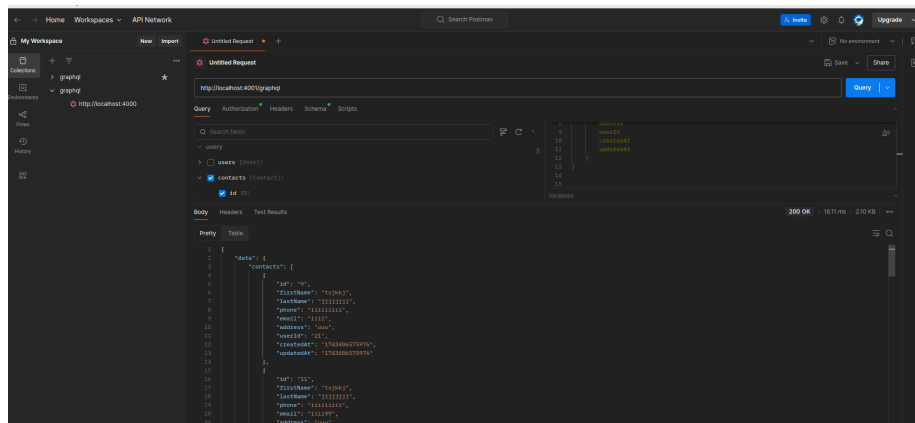
Il est également possible de tester le comportement du token lorsqu'il expire. Si le délai d'expiration défini est dépassé, l'accès aux ressources protégées — comme la liste des contacts — sera automatiquement refusé. Dans ce cas, une erreur de type « Accès refusé » s'affichera.

À noter : Le temps d'expiration d'un token peut être défini en secondes, minutes, heures ou jours, selon le niveau de sécurité souhaité. Il est également

possible de ne pas fixer de durée d'expiration. Dans ce cas, le token restera valide indéfiniment, ce qui peut présenter des risques en cas de compromission.
voici un exemple quand le token est expirer



et voici un exemple de quand l'utilisateur a creer des contact



Fin de la création de notre api GraphQL

À présent que nous avons terminé la création de notre API GraphQL, nous allons passer à son intégration côté client en utilisant React Native afin de la consommer efficacement. Bien entendu, il est possible d'utiliser d'autres technologies côté client, mais dans cet exemple, nous choisissons React Native pour illustrer le processus.

15 Consommation de l'API GraphQL créée

Après avoir utilisé **Apollo Server** pour la création de notre API GraphQL, nous allons désormais utiliser **Apollo Client** avec **React Native** afin de consommer cette API.

Voici les différentes étapes à suivre pour procéder à la consommation de notre API.

16 Consommation de l'API GraphQL créée

Pré-requis : Création d'un projet reactNative

```
1 npx create-expo-app@latest
```

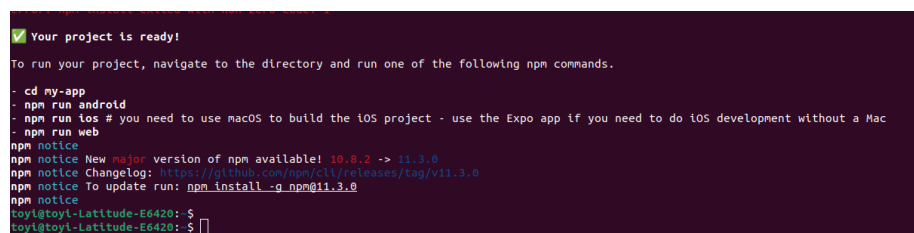


FIGURE 1 – Projet reactNative

Il faut avoir une très bonne connexion et on patiente que tout marche et à la fin de la création du projet on fait :

```
1 cd nomDuProjet
2 // on fait npm install s'il y a eut probl me de connexion
3 npm run reset-project // cela vas vid les donn es venue
  initialement dans le projet
```

On est près maintenant pour consommé notre api.

Installation des dépendances pour graphql : Apolloclient

```
1 npm install @apollo/client graphql
```

Cette bibliothèque apollo client (C'est la bibliothèque principale qui fournit toutes les fonctionnalités nécessaires pour interagir avec une API GraphQL)

nous fournit les fonctionnalités nécessaires pour manipuler les **requêtes GraphQL** y compris le support pour les gql (GraphQL template literals) et la gestion des opérations.

Après avoir terminé l'installation d'**Apollo Client**, nous allons à présent procéder à la configuration de notre client Apollo. Pour cela, nous avons :

Premièrement : Créons un dossier nommé **api**, puis ajoutons-y les fichiers suivants : **apolloclient.ts**, **queries.ts** et **mutations.ts**.

Deuxièmement : Dans le fichier **api/apolloclient.ts**, insérons le code suivant, en veillant à modifier les parties nécessaires selon notre configuration.

```
1  import { ApolloClient, InMemoryCache, createHttpLink } from "@apollo/client";
2  import { setContext } from "@apollo/client/link/context";
3  import AsyncStorage from "@react-native-async-storage/async-storage";
4
5  // Définir le lien HTTP vers notre API GraphQL
6  const httpLink = createHttpLink({
7    uri: "http://localhost:4000/", // Remplacez cette URL par celle
8    // de votre propre serveur GraphQL si elle est différente,
9    // ici nous utilisons l'URL par défaut.
10  });
11
12  // Ici nous avons défini notre middleware pour ajouter le token
13  // d'authentification dans les headers
14  const authLink = setContext(async (_, { headers }) => {
15    // console.log(authLink);
16    const token = await AsyncStorage.getItem("token");
17    // console.log(token);
18    return {
19      headers: {
20        ...headers,
21        authorization: token ? `Bearer ${token}` : "",
22      },
23    };
24  });
25
26  // Créons à présent notre client Apollo
27  const client = new ApolloClient({
28    // A présent on ajoute notre lien de GraphQL et notre middleware
29    link: authLink.concat(httpLink),
30    // Ici on met en cache les résultats des requêtes pour
31    // améliorer les performances de notre API
32    cache: new InMemoryCache(),
33  });
34
35  // Ici on exporte une fois terminée notre objet client d'apollo.
36  export default client;
```

Troisièmement : Dans le fichier **api/queries.ts**, ajoutons le code suivant

en tenant compte du **schéma GraphQL** défini dans notre API.

```
1   import { gql } from '@apollo/client';
2
3   // Renvoie la liste de tout les users
4   export const GET_USERS = gql `
5     query GetUsers {
6       users {
7         id
8         name
9         email
10        contacts {
11          id
12          name
13          phone
14        }
15      }
16    }
17  `;
18
19  // Renvoie un user particulier
20  export const GET_USER = gql `
21    query GetUser($id: Int!) {
22      user(id: $id) {
23        id
24        name
25        email
26        contacts {
27          id
28          name
29          phone
30          address
31        }
32      }
33    }
34  `;
35
36
37
38  //Renvoie l'ensemble de tout les contacts associe e      un user
39
40  export const GET_CONTACTS = gql `
41    query GetContacts {
42      contacts {
43        id
44        name
45        phone
46        address
47        user {
48          id
49          name
50        }
51      }
52    }
53  `;
54
55
```

```

56 // Renvoie un contact bien pr cis avec les informations du user
    qui il appartient .
57 export const GET_CONTACT = gql`
58   query GetContact($id: Int!) {
59     contact(id: $id) {
60       id
61       name
62       phone
63       address
64       user {
65         id
66         name
67         email
68       }
69     }
70   }
71 `;

```

Quatrièmement : Ajoutons le code suivant dans le fichier `api/mutations.ts`, lequel contiendra les requêtes ayant un effet sur notre base de données.

```

1 import { gql } from '@apollo/client';
2
3 // Ici c'est la requ te qui permet de s'enr gistrer sur notre
  plateforme
4 export const SIGNUP = gql`
5   mutation Signup($name: String!, $email: String!, $password:
    String!) {
6     signup(name: $name, email: $email, password: $password) {
7       token
8       user {
9         id
10        name
11        email
12      }
13    }
14  }
15 `;
16
17 // Ici c'est la requ te qui nous permet de se logger sur notre
  paltforme
18 export const LOGIN = gql`
19   mutation Login($email: String!, $password: String!) {
20     login(email: $email, password: $password) {
21       token
22       user {
23         id
24         name
25         email
26       }
27     }
28   }
29 `;
30
31 // Ici c'est la requ te qui permet de cr er un utilisateur
32 export const CREATE_USER = gql`
33   mutation CreateUser($name: String!, $email: String!) {

```



```

34     createUser(name: $name, email: $email) {
35         id
36         name
37         email
38     }
39 }
40 `;
41
42 // Ici c'est la requetes qui permet de cr er un contact
43 export const CREATE_CONTACT = gql`
44     mutation CreateContact($name: String!, $phone: String!, $address:
45         String!, $userId: Int!) {
46         createContact(name: $name, phone: $phone, address: $address,
47             userId: $userId) {
48             id
49             name
50             phone
51             address
52             user {
53                 id
54                 name
55             }
56         }
57     }
58 `;
59
60 // Ici c'est la requete qui permet de mettre jour un
61 // utilisateur
62 export const UPDATE_USER = gql`
63     mutation UpdateUser($id: Int!, $name: String, $email: String) {
64         updateUser(id: $id, name: $name, email: $email) {
65             id
66             name
67             email
68             updatedAt
69         }
70     }
71 `;
72
73 // Ici c'est la requete qui permet de mettre joue un contact
74 export const UPDATE_CONTACT = gql`
75     mutation UpdateContact($id: Int!, $phone: String, $address:
76         String) {
77         updateContact(id: $id, phone: $phone, address: $address) {
78             id
79             phone
80             address
81             updatedAt
82         }
83     }
84 `;
85
86 // Ici c'est la requete qui vas permettre de supprim un
87 // utilisateur
88 export const DELETE_USER = gql`
89     mutation DeleteUser($id: Int!) {
90         deleteUser(id: $id)

```

```

86   }
87   ‘;
88
89   // Ici c’est la requête qui permet de supprimer un contact
90   export const DELETE_CONTACT = gql `
91     mutation DeleteContact($id: Int!) {
92       deleteContact(id: $id)
93     }
94   `;

```

Cinquièmement : À ce stade, nous allons encapsuler notre application avec `ApolloProvider` pour intégrer notre client Apollo, dans lequel nous avons ajouté notre middleware, dans le fichier principal `app/index.tsx`.

```

1  import client from "@api/apolloClient";
2  import DashboardScreen from "@components/auth/DashboardScreen";
3  import { AddContactScreens, Dashboard, HomeScreen, LoginScreen,
4    RegisterScreen } from "@components/screens";
5  import { ApolloProvider } from "@apollo/client";
6  import { createNativeStackNavigator } from "react-navigation/
7    native-stack";
8  import { Text, View } from "react-native";
9
10 const Stack = createNativeStackNavigator();
11
12 export default function Home() {
13   return (
14     <ApolloProvider client={client}>
15       <Stack.Navigator>
16         <Stack.Screen name = "accueil" component={HomeScreen}></
17         Stack.Screen>
18         <Stack.Screen name = "login" component={LoginScreen}></
19         Stack.Screen>
20         <Stack.Screen name = "register" component={RegisterScreen
21         }></Stack.Screen>
22         <Stack.Screen name= "dashboard" component={Dashboard}></
23         Stack.Screen>
24         <Stack.Screen name="AddContact" component={
25         AddContactScreens}></Stack.Screen>
26       </Stack.Navigator>
27       { /* <Text>Je suis L'accueil !</Text> */ }
28     </ApolloProvider>
29   );
30 }

```



FIGURE 2 – Page Welcome

17 Exemple d'utilisation de la requête de login dans notre projet React Native

Il est important de souligner qu'une fois Apollo Client installé, il nous fournit des hooks tels que **useMutation** et **useQuery**, qui nous permettent d'utiliser facilement nos requêtes de mutation et de query. Comme nous allons le voir dans l'exemple suivant :

Fichier LogiScreen.tsx :

```
1 import { useState } from 'react';
2 import { View, Text, TextInput, Button, StyleSheet, Alert } from '
  react-native';
3 import { useMutation } from '@apollo/client';
4 import { LOGIN } from '@api/mutations';
5 import AsyncStorage from '@react-native-async-storage/async-storage
  ';
6
7 const FormulaireLoginScreen = ({ navigation }) => {
8   const [email, setEmail] = useState('');
9   const [password, setPassword] = useState('');
10
11   const [login, { loading, error }] = useMutation(LOGIN);
12
13   const handleLogin = async () => {
14     if (!email || !password) {
15       Alert.alert('Erreur', 'Veuillez remplir tous les champs.');
```

```
16     return;
17   }
18
19   try {
20     const { data } = await login({ variables: { email, password }
21     });
22     // console.log({data});
23     // console.log(data.login.token);
24     // console.log(data.login.user);
25
26     if (data.login.token !== " ") {
27       await AsyncStorage.setItem('token', data.login.token);
28       await AsyncStorage.setItem('user', JSON.stringify(data.
29       login.user));
30       Alert.alert("Connexion réussie", "Bienvenue !");
31       navigation.replace('dashboard');
32     } else {
33       Alert.alert("Erreur", "Identifiants incorrects.");
34     }
35   } catch (err) {
36     console.error(err);
37     Alert.alert("Erreur", error?.message || "Problème de
38     connexion.");
39   }
```

```

40   return (
41     <View style={styles.container}>
42       <Text style={styles.title}>Connexion</Text>
43       <TextInput
44         style={styles.input}
45         placeholder="Email"
46         value={email}
47         onChangeText={setEmail}
48         keyboardType="email-address"
49         autoCapitalize="none"
50         autoComplete={false}
51       />
52       <TextInput
53         style={styles.input}
54         placeholder="Mot de passe"
55         value={password}
56         onChangeText={setPassword}
57         secureTextEntry
58         autoCapitalize="none"
59         autoComplete={false}
60       />
61       <Button title={loading ? 'Chargement...' : 'Se connecter'}
62         onPress={handleLogin} disabled={loading} />
63       {error && <Text style={styles.error}>{error.message}</Text>}
64       <Button title="Pas encore de compte ? S'inscrire" onPress={()
65         => navigation.navigate('register')} />
66     </View>
67   );
68 };
69
70 const styles = StyleSheet.create({
71   container: {
72     flex: 1,
73     justifyContent: 'center',
74     padding: 20,
75     // marginTop: 100
76   },
77   title: {
78     fontSize: 24,
79     marginBottom: 20,
80     textAlign: 'center',
81   },
82   input: {
83     height: 40,
84     borderColor: '#ccc',
85     borderWidth: 1,
86     marginBottom: 12,
87     padding: 10,
88   },
89   error: {
90     color: 'red',
91     marginTop: 10,
92     textAlign: 'center',
93   },
94 });
95
96 export default FormulaireLoginScreen;

```



FIGURE 3 – Page LOGIN

Fichier RegisterScreen.tsx

```

1 import React, { useState } from 'react';
2 import { View, Text, TextInput, Button, StyleSheet, Alert } from '
  react-native';
3 import { useMutation } from '@apollo/client';
4 import { SIGNUP } from '@api/mutations';
5
6 const FormulaireRegisterScreen = ({ navigation }) => {
7   const [name, setName] = useState('');
8   const [email, setEmail] = useState('');
9   const [password, setPassword] = useState('');
10
11   const [signup, { loading, error }] = useMutation(SIGNUP);
12
13   const handleSignup = async () => {
14     try {
15       const { data } = await signup({
16         variables: { name, email, password },
17       });
18
19       if (data.signup.token) {
20         // Sauvegarde du token dans AsyncStorage ou un contexte
21         global
22         Alert.alert("Inscription russie", "Bienvenue !");
23         navigation.replace('login');
24       } catch (err) {
25         console.error(err);
26         Alert.alert("Erreur", "Impossible de cr er le compte.
27         Essayez nouveau.");
28       }
29     };
30
31     return (
32       <View style={styles.container}>
33         <Text style={styles.title}>Inscription</Text>
34         <TextInput
35           style={styles.input}
36           placeholder="Nom"
37           value={name}
38           onChangeText={setName}
39         />
40         <TextInput
41           style={styles.input}
42           placeholder="Email"

```

```

42     value={email}
43     onChangeText={setEmail}
44   />
45   <TextInput
46     style={styles.input}
47     placeholder="Mot de passe"
48     value={password}
49     onChangeText={setPassword}
50     secureTextEntry
51   />
52   <Button title={loading ? 'Chargement...' : 'Sinscrire'}
    onPress={handleSignup} />
53   {error && <Text style={styles.error}>{error.message}</Text>}
54   <Button
55     title="D j un compte ? Se connecter"
56     onPress={() => navigation.navigate('login')}
57   />
58 </View>
59 );
60 };
61
62 const styles = StyleSheet.create({
63   container: {
64     flex: 1,
65     justifyContent: 'center',
66     padding: 20,
67     // marginTop: 100
68   },
69   title: {
70     fontSize: 24,
71     marginBottom: 20,
72     textAlign: 'center',
73   },
74   input: {
75     height: 40,
76     borderColor: '#ccc',
77     borderWidth: 1,
78     marginBottom: 12,
79     paddingLeft: 10,
80   },
81   error: {
82     color: 'red',
83     marginTop: 10,
84   },
85 });
86
87 export default FormulaireRegisterScreen;

```

Fichier DashboardScreen.tsx

```

1   import React, { useEffect, useState } from 'react';
2   import { View, Text, Button, FlatList, StyleSheet, Alert,
    TouchableOpacity, SafeAreaView } from 'react-native';
3   import { useMutation, useQuery } from '@apollo/client';
4   import AsyncStorage from '@react-native-async-storage/async-storage';
5   import { GET_USER } from '@api/queries';

```

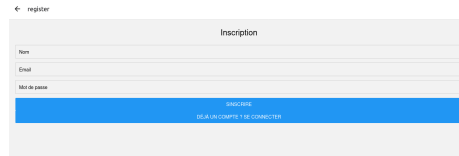


FIGURE 4 – Page REGISTER

```

6 import { DELETE_CONTACT } from '@api/mutations';
7
8 const DashboardScreen = ({ navigation }) => {
9   const [userId, setUserId] = useState(null);
10
11   const { data, loading, error, refetch } = useQuery(GET_USER, {
12     variables: { id: userId },
13     skip: !userId,
14   });
15
16
17   useEffect(() => {
18     const fetchUser = async () => {
19       const token = await AsyncStorage.getItem('token');
20       if (token) {
21         const storedUser = await AsyncStorage.getItem('user');
22         if (storedUser) {
23           const user = JSON.parse(storedUser);
24           setUserId(user.id);
25         }
26       } else {
27         Alert.alert('Erreur', 'Vous devez être connecté pour accéder au Dashboard');
28         navigation.replace('login');
29       }
30     };
31     fetchUser();
32   }, [navigation]);
33
34   const [deleteContact] = useMutation(DELETE_CONTACT, {
35     onCompleted: () => {
36       Alert.alert('Succès', 'Contact supprimé avec succès');
37       refetch();
38     },
39     onError: (error) => {
40       Alert.alert('Erreur', error.message);
41     }
42   });
43
44   const handleDelete = (contactId) => {
45     console.log("ID du contact à supprimer :", contactId);
46     Alert.alert('Confirmation', 'Voulez-vous vraiment supprimer ce contact ?', [
47       { text: 'Annuler', style: 'cancel' },
48       {
49         text: 'Supprimer',

```

```

51     onPress: async () => {
52       try {
53         const { data } = await deleteContact({
54           variables: { id: contactId },
55         });
56         console.log("R ponse de suppression:", data);
57       } catch (err) {
58         console.error("Erreur lors de la suppression:", err);
59         Alert.alert('Erreur', err.message);
60       }
61     }
62   }
63   });
64 };
65
66 const handleLogout = async () => {
67   await AsyncStorage.removeItem('token');
68   await AsyncStorage.removeItem('user');
69   navigation.replace('login');
70 };
71
72 if (loading) return <Text>Chargement...</Text>;
73 if (error) return <Text>Erreur : {error.message}</Text>;
74
75 return (
76   <SafeAreaView style={styles.container}>
77     {data?.user ? (
78       <>
79         <Text style={styles.title}>Bienvenue, {data.user.name}
80         !</Text>
81         <Text style={styles.info}>Email : {data.user.email}</Text>
82       </>
83       <View style={styles.buttonContainer}>
84         <Button title="Ajouter un Contact" onPress={() =>
85           navigation.navigate('AddContact')} />
86         <View style={styles.space} />
87         <Button title="Se D connecter" onPress={handleLogout}
88         color="red" />
89       </View>
90       <Text style={styles.subtitle}>      Mes Contacts :</Text>
91     ) : (
92       <Text style={styles.subtitle}>      Mes Contacts :</Text>
93     )
94   }
95   {data?.user?.contacts?.length > 0 ? (
96     <FlatList
97       data={data.user.contacts}
98       keyExtractor={(contact) => contact.id.toString()}
99       contentContainerStyle={{ flexGrow: 1 }}
100       renderItem={({ item }) => (
101         <View style={styles.contactItem}>
102           <Text style={styles.contactName}>      {item.
103             name}</Text>
104           <Text style={styles.contactPhone}>      {item.
105             phone}</Text>
106           <Text style={styles.contactAddress}>      {item.
107             address}</Text>
108           <View style={styles.buttonGroup}>

```



```

100         <TouchableOpacity style={[styles.button, styles
.updateButton]} onPress={() => navigation.navigate('
UpdateContact', { contact: item })}>
101         <Text style={styles.buttonText}>Modifier</
Text>
102         </TouchableOpacity>
103         <TouchableOpacity style={[styles.button, styles
.deleteButton]} onPress={() => handleDelete(item.id)}>
104         <Text style={styles.buttonText}>Supprimer</
Text>
105         </TouchableOpacity>
106         <TouchableOpacity style={[styles.button, styles
.showButton]} onPress={() => navigation.navigate('ShowContact',
{ contact: item })}>
107         <Text style={styles.buttonText}>Voir</Text>
108         </TouchableOpacity>
109     </View>
110 </View>
111     )}
112 </>
113 ) : (
114     <Text style={styles.noContacts}>Aucun contact trouv
.</Text>
115     )}
116 </>
117 ) : (
118     <Text>Utilisateur non trouv </Text>
119     )}
120 </SafeAreaView>
121 );
122 };
123
124
125 const styles = StyleSheet.create({
126     container: { flex: 1, padding: 20, backgroundColor: '#fff' },
127     title: { fontSize: 24, fontWeight: 'bold', marginBottom: 10,
        textAlign: 'center' },
128     info: { fontSize: 18, marginBottom: 20, textAlign: 'center' },
129     subtitle: { fontSize: 20, fontWeight: 'bold', marginTop: 20 },
130     contactItem: { backgroundColor: '#f8f9fa', padding: 10,
        borderRadius: 8, marginVertical: 5 },
131     contactName: { fontSize: 16, fontWeight: 'bold' },
132     contactPhone: { fontSize: 14, color: '#555' },
133     contactAddress: { fontSize: 14, color: '#555' },
134     noContacts: { fontSize: 16, color: '#888', textAlign: 'center',
        marginTop: 10 },
135     buttonContainer: { flexDirection: 'row', justifyContent: 'space-
        between', marginTop: 20 },
136     buttonGroup: { flexDirection: 'row', justifyContent: 'space-
        between', marginTop: 10 },
137     button: { flex: 1, padding: 8, borderRadius: 5, alignItems: '
        center', marginHorizontal: 5 },
138     updateButton: { backgroundColor: '#007bff' },
139     deleteButton: { backgroundColor: '#dc3545' },
140     showButton: { backgroundColor: '#28a745' },
141     buttonText: { color: 'white', fontWeight: 'bold' },
142     space: { width: 20 },

```

```
143 });  
144  
145 export default DashboardScreen;
```

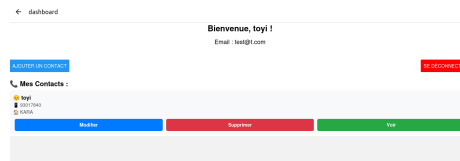


FIGURE 5 – Page DASHBOARD

Fin de la partie consommation de l'API GraphQL